



# HUST

## Graduation Research 2

---

# Hệ thống Trợ lý Y tế Đa phương thức sử dụng Multi-Agent

---

**Author:** Hoàng Quốc Hưng

hung.hq226043@sis.hust.edu.vn

**Supervisor:** PhD. Dinh Viet Sang

**Department:** Department of Computer Science

**School:** School of Information and Communication Technology

Hanoi, Vietnam  
January 2026

# Abstract

This thesis proposes a Multi-Agent Medical Assistant whose main objective is to improve the reliability, grounding, and continuity of medical consultation across text and image inputs. Instead of relying on a single generative model, the system is designed to route user requests to specialized agents, combine structured and unstructured medical evidence, identify when the available clinical context is insufficient, and preserve useful patient context across sessions without treating it as verified diagnosis. For text-based consultation, the system coordinates Retrieval-Augmented Generation, Knowledge Graph reasoning, multi-hop clarification, MedGemma CoT-based information sufficiency assessment, and Web Search fallback to reduce unsupported or premature medical responses. For vision-based consultation, the system integrates colonoscopy polyp segmentation with Polyp Visual Question Answering, where the segmentation mask acts as a spatial grounding prior while the original endoscopic image remains the source of visual evidence. The system also incorporates patient-scoped semantic memory and human-in-the-loop validation to support personalization, feedback collection, and safer review. Experimental results show that the proposed design achieves several system-level goals: Knowledge Graph evidence improves relation-oriented text reasoning, post-processing improves segmentation consistency for downstream use, LoRA fine-tuning substantially improves Polyp VQA performance, and segmentation grounding improves judge-based VQA quality on the mask-matched subset. Overall, the thesis demonstrates that a modular, grounded, memory-aware architecture can improve observability, evidence control, multimodal reasoning, and safety boundaries in a medical assistant intended for decision support rather than definitive clinical diagnosis.

**Keywords:** Multi-agent medical assistant, retrieval-augmented generation, knowledge graph, medical visual question answering, visual grounding, long-term memory, human-in-the-loop validation.

# Mục lục

|          |                                                                                                    |           |
|----------|----------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Chapter 1. Introduction</b>                                                                     | <b>7</b>  |
| 1.1      | Motivation . . . . .                                                                               | 7         |
| 1.2      | Problem Statement . . . . .                                                                        | 7         |
| 1.3      | Technical Challenges . . . . .                                                                     | 7         |
| 1.4      | Objectives . . . . .                                                                               | 8         |
| 1.5      | Contributions . . . . .                                                                            | 8         |
| 1.6      | Research Questions . . . . .                                                                       | 9         |
| 1.7      | Scope of the Thesis . . . . .                                                                      | 9         |
| <b>2</b> | <b>Chapter 2. Background and Related Work</b>                                                      | <b>9</b>  |
| 2.1      | Large Language and Vision-Language Models in Healthcare . . . . .                                  | 9         |
| 2.2      | Medical Question Answering and Multi-hop Reasoning . . . . .                                       | 10        |
| 2.3      | Retrieval-Augmented Generation for Medical QA . . . . .                                            | 11        |
| 2.4      | Medical Knowledge Graphs . . . . .                                                                 | 11        |
| 2.5      | Multi-agent AI Systems . . . . .                                                                   | 12        |
| 2.6      | LoRA and Parameter-Efficient Fine-tuning . . . . .                                                 | 12        |
| 2.7      | Polyp Segmentation and Neoplasm Characterization . . . . .                                         | 12        |
| 2.8      | Long-Term Memory for AI Agents . . . . .                                                           | 13        |
| 2.9      | Human-in-the-loop for Medical AI . . . . .                                                         | 14        |
| <b>3</b> | <b>Chapter 3. Proposed Multi-Agent Medical Assistant Architecture</b>                              | <b>14</b> |
| 3.1      | Overall Architecture . . . . .                                                                     | 14        |
| 3.2      | LangGraph-based Agent Orchestration . . . . .                                                      | 15        |
| 3.3      | Agent Cooperation and Cross-module Workflows . . . . .                                             | 16        |
| 3.4      | Decision Agent . . . . .                                                                           | 17        |
| 3.5      | Knowledge and Context Sources . . . . .                                                            | 18        |
| 3.6      | Safety and Human-in-the-loop Validation . . . . .                                                  | 19        |
| <b>4</b> | <b>Chapter 4. Text Module: Retrieval and Reasoning</b>                                             | <b>19</b> |
| 4.1      | Role and Processing Flow of the Text Module . . . . .                                              | 19        |
| 4.2      | Text Query Types and Routing Behavior . . . . .                                                    | 21        |
| 4.3      | Conversation Agent and Multi-hop QA . . . . .                                                      | 21        |
| 4.3.1    | Single-turn Limitation versus Multi-hop Clarification . . . . .                                    | 22        |
| 4.3.2    | Illustrative Multi-hop Consultation Flow . . . . .                                                 | 23        |
| 4.4      | Clinical Context Requirements before MedGemma CoT . . . . .                                        | 24        |
| 4.5      | Medical Knowledge Retrieval with RAG and Knowledge Graph . . . . .                                 | 25        |
| 4.5.1    | Data Retrieval Layer . . . . .                                                                     | 25        |
| 4.5.2    | KG Agent . . . . .                                                                                 | 26        |
| 4.5.3    | RAG Agent . . . . .                                                                                | 26        |
| 4.5.4    | Coordinated KG-RAG Evidence Construction . . . . .                                                 | 27        |
| 4.6      | Parallel Reasoning and Fallback Strategy . . . . .                                                 | 28        |
| 4.7      | MedGemma CoT for Medical Decision Support . . . . .                                                | 28        |
| 4.8      | Safe Medical Response Generation . . . . .                                                         | 30        |
| <b>5</b> | <b>Chapter 5. Vision Module: General Medical Image Analysis, Polyp Segmentation, and Polyp VQA</b> | <b>31</b> |
| 5.1      | Role and Processing Flow of the Vision Module . . . . .                                            | 31        |
| 5.2      | Image Routing and Visual Task Classification . . . . .                                             | 32        |
| 5.3      | General Medical Image Agent . . . . .                                                              | 32        |

|           |                                                                       |           |
|-----------|-----------------------------------------------------------------------|-----------|
| 5.4       | Polyp Segmentation Agent . . . . .                                    | 33        |
| 5.5       | DeepLabV3+ for Polyp Segmentation . . . . .                           | 33        |
| 5.6       | Post-processing and Overlay Mask . . . . .                            | 35        |
| 5.7       | Polyp VQA Agent Design with LoRA Fine-tuned VLMs . . . . .            | 37        |
| 5.8       | Segmentation-to-VQA Agent Cooperation . . . . .                       | 38        |
| 5.8.1     | Example Interaction Pattern . . . . .                                 | 39        |
| 5.9       | Architectural Mechanism of Grounded Polyp VQA . . . . .               | 40        |
| 5.10      | Original-image and Segmentation-grounded VQA Settings . . . . .       | 41        |
| <b>6</b>  | <b>Chapter 6. Memory Layer: Long-Term Context for Medical Agents</b>  | <b>42</b> |
| 6.1       | Role of the Memory Layer . . . . .                                    | 42        |
| 6.2       | Integration Architecture . . . . .                                    | 42        |
| 6.3       | Technical Pipeline . . . . .                                          | 43        |
| 6.4       | Impact on Agent Reasoning . . . . .                                   | 45        |
| 6.5       | Implementation Correspondence . . . . .                               | 46        |
| 6.6       | Safety Constraints and Limitations . . . . .                          | 47        |
| <b>7</b>  | <b>Chapter 7. Data and Experimental Setup</b>                         | <b>47</b> |
| 7.1       | VimedAQA Dataset . . . . .                                            | 47        |
| 7.2       | VietMedKG . . . . .                                                   | 48        |
| 7.3       | BKAI-IGH NeoPolyp-Small Dataset . . . . .                             | 49        |
| 7.4       | Kvasir-VQA-X1 Dataset . . . . .                                       | 49        |
| 7.5       | Polyp VQA Fine-tuning Setup . . . . .                                 | 50        |
| 7.6       | Memory Evaluation Scenarios . . . . .                                 | 52        |
| 7.7       | Evaluation Metrics . . . . .                                          | 52        |
| 7.8       | Experimental Protocol . . . . .                                       | 52        |
| <b>8</b>  | <b>Chapter 8. Results and Analysis</b>                                | <b>53</b> |
| 8.1       | Text Module Results and Analysis . . . . .                            | 53        |
| 8.2       | Polyp Segmentation Results and Analysis . . . . .                     | 53        |
| 8.3       | Polyp VQA Fine-tuning Results and Analysis on Kvasir-VQA-X1 . . . . . | 54        |
| <b>9</b>  | <b>Chapter 9. Overall Discussion and Limitations</b>                  | <b>56</b> |
| 9.1       | Strengths of the Proposed System . . . . .                            | 56        |
| 9.2       | Text Module Limitations . . . . .                                     | 56        |
| 9.3       | Vision Module Limitations . . . . .                                   | 57        |
| 9.4       | Kvasir-VQA-X1 Limitations and Domain Transfer . . . . .               | 57        |
| 9.5       | Memory Layer Limitations . . . . .                                    | 57        |
| 9.6       | Evaluation and Safety Limitations . . . . .                           | 58        |
| <b>10</b> | <b>Chapter 10. Conclusion and Future Work</b>                         | <b>58</b> |
| 10.1      | Summary of Achievements . . . . .                                     | 58        |
| 10.2      | Technical Contributions . . . . .                                     | 58        |
| 10.3      | Future Work . . . . .                                                 | 59        |

## List of Tables

|   |                                                                                   |    |
|---|-----------------------------------------------------------------------------------|----|
| 1 | Benefits of the multi-agent design illustrated by the detailed workflow . . . . . | 16 |
| 2 | Main cooperation patterns in the proposed multi-agent architecture . . . . .      | 17 |
| 3 | Decision Agent routing behavior . . . . .                                         | 18 |

|    |                                                                                                             |    |
|----|-------------------------------------------------------------------------------------------------------------|----|
| 4  | Knowledge and context sources with reliability considerations . . . . .                                     | 19 |
| 5  | Text query types and corresponding routing behavior . . . . .                                               | 21 |
| 6  | Coordinated evidence construction in the Text Module . . . . .                                              | 28 |
| 7  | MedGemma CoT decision-support stages . . . . .                                                              | 30 |
| 8  | Complementary roles of the Polyp Segmentation Agent and Polyp VQA Agent .                                   | 39 |
| 9  | Mapping between the Memory Layer design and the current implementation . .                                  | 46 |
| 10 | Local VimedAQA record distribution used for RAG indexing . . . . .                                          | 48 |
| 11 | Kvasir-VQA-X1 training data before and after segmentation-mask filtering . . .                              | 50 |
| 12 | LoRA fine-tuning hyperparameters for Polyp VQA . . . . .                                                    | 51 |
| 13 | Text Module benchmark results . . . . .                                                                     | 53 |
| 14 | Dice Score comparison for polyp segmentation models . . . . .                                               | 54 |
| 15 | Polyp VQA evaluation on Kvasir-VQA-X1 . . . . .                                                             | 55 |
| 16 | LLM Judge comparison between LoRA-only and visual-grounded MedGemma<br>on the mask-matched subset . . . . . | 55 |

## List of Figures

|    |                                                                                                |    |
|----|------------------------------------------------------------------------------------------------|----|
| 1  | Overall architecture of the Multi-Agent Medical Assistant . . . . .                            | 14 |
| 2  | Detailed agent workflow and module coordination . . . . .                                      | 15 |
| 3  | Text Module architecture for medical knowledge retrieval . . . . .                             | 20 |
| 4  | Traditional single-turn approach with limited clinical context . . . . .                       | 22 |
| 5  | Multi-hop QA approach with iterative clarification and safer response refinement               | 23 |
| 6  | Parallel KG-RAG processing in the Text Module . . . . .                                        | 25 |
| 7  | Medical chain-of-thought and information sufficiency flow . . . . .                            | 29 |
| 8  | Vision Module workflow for medical image analysis . . . . .                                    | 32 |
| 9  | DeepLabV3+ encoder-decoder architecture with Atrous Spatial Pyramid Pooling                    | 34 |
| 10 | Visualization of raw and post-processed polyp segmentation masks . . . . .                     | 37 |
| 11 | Medical QA dataset overview used for retrieval-oriented experiments . . . . .                  | 47 |
| 12 | Overview of the VietMedKG data source used by the Knowledge Graph Agent .                      | 48 |
| 13 | Example images and annotations from the BKAI-IGH NeoPolyp-Small dataset .                      | 49 |
| 14 | Example structure of the Kvasir-VQA-X1 dataset for gastrointestinal endoscopy<br>VQA . . . . . | 50 |

# Preface

The increasing demand for reliable, accessible, and personalized medical information motivates the development of intelligent medical assistant systems. Users often describe symptoms in incomplete or informal language, search for medical information from sources of uneven quality, and may need support across multiple interactions rather than a single isolated conversation.

Recent advances in language models, retrieval-augmented generation, knowledge graphs, vision-language models, and agent orchestration provide an opportunity to build medical AI systems that are more grounded, modular, and context-aware. At the same time, medical AI remains a safety-sensitive research problem. A useful assistant should not only generate fluent responses, but should also identify insufficient clinical context, ground its reasoning in appropriate evidence, handle multimodal inputs cautiously, and preserve patient context without converting self-reported information into verified diagnosis.

This thesis presents a Multi-Agent Medical Assistant designed to study these issues at the system architecture level. The work focuses on how specialized agents can cooperate through structured and unstructured evidence, visual grounding, patient-scoped memory, and human-in-the-loop validation. The thesis does not aim to build a deployable clinical product or replace professional medical judgment. Instead, it investigates the technical feasibility, design trade-offs, and evaluation behavior of a modular medical assistant intended for decision support.

## Summary of Content and Contributions

This thesis makes the following main contributions:

- **A modular multi-agent architecture for medical assistance:** The system separates routing, text reasoning, vision reasoning, memory retrieval, and validation into specialized components. This improves observability and allows intermediate evidence to be inspected before final response generation.
- **A grounded Text Module for medical consultation:** The text workflow coordinates multi-hop clarification, query expansion, Retrieval-Augmented Generation, Knowledge Graph reasoning, MedGemma CoT-based information sufficiency assessment, and Web Search fallback. This design distinguishes evidence availability from clinical sufficiency and reduces premature answer generation.
- **A segmentation-grounded Vision Module for Polyp VQA:** The vision workflow connects colonoscopy polyp segmentation with visual question answering. The segmentation mask is formulated as a spatial grounding prior, while the original endoscopic image remains the source of appearance-level visual evidence.
- **A patient-scoped memory layer for contextual continuity:** The thesis integrates Mem0-style semantic memory to retrieve durable patient-reported context across sessions. Memory is used as supporting context for routing and personalization, not as authoritative clinical truth.
- **An experimental evaluation of component and system behavior:** The thesis evaluates text reasoning with One-Hop Accuracy and Multi-Hop Awareness Score, segmentation with Dice Score, and Polyp VQA with BLEU-4, ROUGE-L, and LLM-as-a-judge criteria. The results analyze not only model performance, but also how evidence grounding, post-processing, LoRA fine-tuning, and segmentation guidance affect system-level reliability.

# 1 Chapter 1. Introduction

## 1.1 Motivation

Healthcare information seeking is increasingly mediated by digital systems, yet most users still face a fundamental gap between their symptoms and reliable medical interpretation. A patient may describe a symptom in informal language, omit clinically important details, misunderstand medication risks, or combine several concerns across different interaction sessions. At the same time, medical information available on the internet is abundant but uneven in quality, difficult to verify, and often not adapted to the user’s clinical context.

Recent progress in large language models (LLMs), retrieval-augmented generation (RAG), knowledge graphs, and vision-language models (VLMs) makes it possible to design medical assistants that can reason over both text and images. However, a direct application of a single generative model is not sufficient for a safety-critical domain. Medical consultation requires source grounding, uncertainty awareness, follow-up questioning, multimodal interpretation, and mechanisms for human validation.

This thesis develops a **Multi-Agent Medical Assistant** that combines text-based medical reasoning, medical image analysis, long-term memory, and human-in-the-loop validation. The system is designed as a coordinated set of specialized agents rather than a single monolithic model. In the text pathway, structured Knowledge Graph evidence and unstructured RAG evidence are coordinated before MedGemma CoT performs information-sufficiency assessment. In the vision pathway, polyp segmentation provides spatial grounding for VQA rather than replacing direct interpretation of the original endoscopic image. The memory layer supplies patient-scoped contextual continuity while remaining separate from verified clinical diagnosis.

## 1.2 Problem Statement

The problem addressed in this thesis is how to design and evaluate a medical assistant that remains grounded, modular, and cautious across heterogeneous user inputs. The system must not only generate answers, but also decide when evidence is sufficient, when follow-up information is required, and how text, image, memory, and validation signals should be coordinated. Concretely, the thesis investigates whether a multi-agent architecture can:

- produce medical answers grounded in structured and unstructured evidence;
- detect incomplete clinical context and collect missing information through multi-hop questioning;
- coordinate RAG, Knowledge Graph reasoning, MedGemma CoT sufficiency assessment, and Web Search fallback for text-based consultation;
- connect general medical image analysis with a specialized colonoscopy polyp workflow;
- use polyp segmentation masks as spatial grounding for VQA while preserving the original image as the visual evidence source;
- maintain patient-scoped context across sessions without treating memory as verified diagnosis;
- support human-in-the-loop validation for review and safety feedback.

The focus is not on building a web application interface, but on the AI architecture, reasoning workflow, multimodal coordination, memory integration, and experimental analysis.

## 1.3 Technical Challenges

Several challenges motivate the proposed architecture.

**Incomplete clinical context.** Users often provide insufficient symptom descriptions. A

reliable medical assistant should recognize when information is missing and ask follow-up questions before generating a recommendation.

**Hallucination and source reliability.** LLMs can generate fluent but unsupported medical content. Grounding responses in RAG documents, Knowledge Graph facts, and trusted external sources reduces this risk. However, retrieved evidence alone is not sufficient; the system must also decide whether the user’s clinical context is complete enough for a safe answer.

**Structured and unstructured medical knowledge.** Medical reasoning often requires both detailed textual explanation and structured relations between symptoms, diseases, drugs, and risk factors. RAG and Knowledge Graphs provide complementary capabilities.

**Medical image understanding.** Medical image analysis requires models that can identify visual patterns and communicate findings carefully. In colonoscopy images, polyp localization and neoplasm characterization are important subtasks.

**Visual grounding for VQA.** A VLM answering questions from an image may attend to irrelevant regions. Segmentation masks can guide the VQA model toward clinically relevant regions by providing a spatial prior, but the final answer must still be derived from visual evidence in the original image. Incorrect masks may also introduce noise or misdirect the VQA model.

**Long-term patient context.** Medical interaction is rarely isolated. Information such as allergies, medication use, chronic diseases, or repeated symptoms may remain relevant across sessions. A memory layer must preserve useful facts without treating them as official diagnoses.

**Safety and validation.** Medical AI systems require feedback mechanisms and explicit boundaries. Human-in-the-loop validation helps collect feedback and identify responses that may require review.

## 1.4 Objectives

The main objective is to design and evaluate an AI-centered multi-agent medical assistant with four core capabilities:

- **Text Module:** medical reasoning with Conversation Agent, Multi-hop QA, query expansion, RAG, Knowledge Graph, MedGemma CoT sufficiency assessment, Web Search, and safe response generation;
- **Vision Module:** general image analysis, polyp segmentation, and grounded Polyp VQA;
- **Memory Layer:** patient-scoped long-term context using Mem0-style memory integration;
- **Human Validation:** user validation and feedback logging as a human-in-the-loop safety mechanism.

## 1.5 Contributions

The main contributions of this thesis are organized around architecture, grounding, and evaluation:

- a modular multi-agent architecture that separates routing, text reasoning, vision reasoning, memory retrieval, and human validation while allowing intermediate evidence to be shared across agents;
- a grounded Text Module that coordinates Multi-hop QA, query expansion, RAG, Knowledge Graph reasoning, MedGemma CoT sufficiency assessment, and fallback search to reduce unsupported or premature medical responses;
- a Vision Module that connects general medical image analysis, polyp segmentation, and segmentation-grounded Polyp VQA for multimodal reasoning;



- a polyp segmentation pipeline with connected-component post-processing and region-wise color dominance refinement to improve mask consistency for interpretation and downstream VQA grounding;
- a grounded Polyp VQA formulation that adapts MedGemma-4B-it and Qwen3.5-4B with LoRA and compares original-image with mask-guided inputs;
- a patient-scoped long-term memory layer using Mem0 to support contextual continuity while keeping stored patient-reported facts separate from verified diagnoses;
- an evaluation design that analyzes both component performance and system behavior using Dice Score, BLEU-4, ROUGE-L, LLM Judge, One-Hop Accuracy, Multi-Hop Awareness Score, and memory-oriented scenarios.

## 1.6 Research Questions

This thesis is guided by five research questions. First, how can a medical assistant combine conversational reasoning, RAG, Knowledge Graph reasoning, and Web Search to reduce unsupported medical responses? Second, how can information sufficiency assessment inside the MedGemma CoT workflow improve safety in symptom-based consultation? Third, can segmentation masks improve Polyp VQA by providing visual grounding for a fine-tuned VLM? Fourth, how can long-term memory support personalization without turning patient-reported facts into verified diagnoses? Finally, what trade-offs appear between modularity, response quality, latency, and safety in a multi-agent medical assistant?

These questions connect the architecture with the evaluation methodology. The thesis therefore evaluates the system as a reasoning architecture and not as a clinical product ready for deployment.

## 1.7 Scope of the Thesis

The thesis focuses on AI methods and system-level reasoning rather than user-interface engineering. Its main contribution lies in agent orchestration, knowledge retrieval, multimodal reasoning, long-term memory integration, and validation-oriented safety.

The medical scope is also limited. The system is intended to provide information support and preliminary reasoning. It does not replace physicians, does not provide definitive diagnosis, and does not make treatment decisions. For high-risk symptoms or uncertain findings, the system should recommend professional medical consultation.

# 2 Chapter 2. Background and Related Work

## 2.1 Large Language and Vision-Language Models in Healthcare

Large language models have become a central direction in medical AI because they can represent clinical knowledge, answer medical questions, and generate patient-facing explanations [12]. Med-PaLM and Med-PaLM 2 show that instruction tuning, medical-domain adaptation, and prompting strategies can substantially improve medical question answering [13]. Open medical LLMs such as PMC-LLaMA and BioMistral follow another direction by adapting general-purpose models to biomedical literature, textbooks, and medical instruction data [14, 15]. These studies show clear progress, but they mostly evaluate the model as an independent QA system rather than as part of a complete consultation workflow.

Medical vision-language models extend this trend from text-only reasoning to image-text reasoning. LLaVA-Med and Med-Flamingo adapt multimodal foundation models for biomedical image conversation, VQA, and rationale generation [16, 17]. More recent models such as MedGemma provide medical vision-language backbones, while general model families such as

Qwen offer strong multilingual and instruction-following capabilities that can be adapted to medical tasks [40, 41]. In gastrointestinal endoscopy, datasets such as Kvasir-VQA and Kvasir-VQA-X1 further indicate a shift from image classification toward question answering over clinical visual evidence [37, 38].

However, current LLMs and VLMs still have important limitations for medical assistants. They can generate fluent but unsupported answers, especially when the user question lacks clinical context or when the model relies only on parametric memory [18]. Many models also operate without explicit retrieval, Knowledge Graph evidence, segmentation grounding, or human validation. For VLMs, correct-looking answers may come from dataset priors or textual shortcuts rather than from the relevant image region, which is risky in endoscopy because lesion interpretation depends on local boundary, texture, morphology, and color patterns.

This thesis therefore treats LLMs and VLMs as components inside a larger safety-aware architecture. The proposed system combines model generation with routing, RAG, Knowledge Graph reasoning, MedGemma CoT-based information sufficiency assessment, patient-scoped memory, segmentation-based visual grounding, and human validation. This design addresses the limitation of standalone models by testing whether external evidence and spatial grounding improve reasoning reliability rather than only answer fluency.

## 2.2 Medical Question Answering and Multi-hop Reasoning

Medical question answering has traditionally been evaluated through benchmark datasets such as PubMedQA, MedQA, and MedMCQA [19, 20, 21]. These datasets are valuable because they test biomedical reading comprehension, medical exam knowledge, and multiple-choice clinical reasoning. Vietnamese medical QA datasets such as VimedAQA further show the importance of language-specific resources for local healthcare applications [8]. However, these benchmarks usually provide a well-formed question and a fixed answer space. Real users often provide incomplete, informal, or ambiguous descriptions, so benchmark performance alone is not sufficient to evaluate a consultation-oriented assistant.

The key difference is that medical consultation is frequently underspecified at the first turn. A user asking about chest pain, abdominal pain, fever, dizziness, or medication use may omit onset time, severity, duration, associated symptoms, age, pregnancy status, allergies, chronic diseases, or current medication. A direct answer may therefore be clinically incomplete even if it is linguistically fluent. This creates a gap between single-turn medical QA and interactive medical reasoning.

Multi-hop reasoning addresses this gap by decomposing a complex question into intermediate reasoning steps and evidence requirements. General multi-hop QA research shows that answering some questions requires connecting multiple supporting facts rather than retrieving one isolated passage [22]. In the medical domain, the same principle appears in two forms. The first form is evidence multi-hop reasoning, where the system combines information from multiple sources such as retrieved documents, Knowledge Graph relations, and previous conversation context. The second form is dialogue multi-hop reasoning, where the system asks follow-up questions before generating a final response.

This thesis focuses on the second form as a safety mechanism and uses the first form for evidence synthesis. The Conversation Agent helps identify missing clinical variables, while KG and RAG agents construct structured and unstructured evidence when the query is answerable from available sources. MedGemma CoT then acts as an information-sufficiency decision point: it decides whether the system should synthesize an answer, ask a targeted follow-up question, or activate fallback search. This workflow makes multi-hop QA more than a prompting technique; it becomes a control mechanism for deciding when to answer, when to retrieve, when to ask, and when to recommend professional care. It also separates general medical education from patient-

specific advice: a question such as “What is gastritis?” can often be answered from retrieved explanatory material, whereas “Do I have gastritis?” requires additional clinical variables and a more cautious response.

## 2.3 Retrieval-Augmented Generation for Medical QA

Retrieval-Augmented Generation combines neural generation with external document retrieval [9]. In a medical assistant, RAG can reduce hallucination by grounding the model response in retrieved medical documents. A typical RAG pipeline includes query expansion, embedding-based retrieval, document reranking, and response generation with source context.

The proposed system uses RAG for Vietnamese medical question answering. Documents are embedded and stored in a vector database, while the response generator uses retrieved context to produce structured answers. This approach is useful when the answer requires explanatory text, treatment descriptions, disease overviews, or patient-friendly medical information.

RAG is not sufficient by itself. Retrieval can fail when a query is underspecified, when relevant documents are absent from the corpus, or when semantically similar but clinically irrelevant passages are retrieved. A retrieved passage can also make an answer appear grounded even when the original clinical question remains incomplete. For this reason, the proposed Text Module combines RAG with query expansion, reranking, context filtering, Knowledge Graph reasoning, and MedGemma CoT sufficiency assessment.

Medical RAG also requires attention to language and domain mismatch. In Vietnamese medical QA, users may mix colloquial expressions with formal medical terms. A retrieval system must bridge this gap without retrieving unrelated passages. Query expansion and filtering are therefore not auxiliary features; they are necessary parts of making retrieval usable in real consultation-style interactions.

## 2.4 Medical Knowledge Graphs

Knowledge Graphs represent medical knowledge as entities and relations. Entities may include diseases, symptoms, drugs, body parts, risk factors, tests, or treatments. Relations encode structured links such as disease-has-symptom, drug-treats-disease, or disease-associated-with-risk-factor.

Compared with RAG, a Knowledge Graph is more suitable for structured reasoning. It can support explicit traversal of disease-symptom relationships and provide more interpretable evidence for certain diagnostic suggestions. In this thesis, the Knowledge Graph Agent queries VietMedKG through Neo4j and supports reasoning for symptom-to-disease and related medical questions.

Knowledge Graphs also have limitations. They depend on relation coverage and graph construction quality. A graph may encode important disease-symptom relations but lack detailed explanation or updated guidance. Therefore, this thesis treats KG and RAG as complementary rather than competing approaches.

The complementarity is especially relevant for medical reasoning. A Knowledge Graph can provide a compact relation such as a symptom-disease link, but a patient-facing answer often requires explanation, uncertainty, and practical next steps. RAG can provide that explanatory material. Conversely, RAG may retrieve long text but fail to make an explicit relation clear. KG can provide structure to support the final synthesis. In the proposed system, these two evidence channels are not treated as final answers; they are passed to the reasoning stage, where sufficiency, uncertainty, and safety boundaries are assessed before response generation.

## 2.5 Multi-agent AI Systems

Multi-agent systems divide complex tasks among specialized agents. In medical AI, this is useful because different queries require different capabilities. A symptom consultation may require multi-hop questioning and sufficiency assessment, a disease explanation may require RAG, a structured reasoning query may require Knowledge Graph traversal, an image question may require a vision-language model, and a follow-up question may require patient memory.

The proposed system uses a Decision Agent to route user requests to the appropriate agents. The workflow is orchestrated with LangGraph [1], which allows the system to maintain short-term state, coordinate agent execution, retrieve patient memory before routing, and route to fallback strategies when internal knowledge is insufficient.

The multi-agent design also improves observability. The system can identify which agent handled a query and which source of information was used. This is useful for debugging, evaluation, and user trust in safety-sensitive medical applications.

However, multi-agent systems introduce coordination challenges. Routing errors, inconsistent outputs between agents, and increased latency can reduce system quality. A medical multi-agent assistant must therefore include clear routing rules, fallback behavior, and evaluation of intermediate decisions. This thesis treats agent orchestration as a core technical contribution rather than as an implementation detail.

## 2.6 LoRA and Parameter-Efficient Fine-tuning

Fine-tuning large VLMs is computationally expensive because full-parameter updates require significant memory and training resources. LoRA provides a parameter-efficient alternative by learning low-rank adaptation matrices while keeping most original model parameters fixed [39]. This reduces the training cost and makes domain adaptation more practical.

In this thesis, LoRA is used to adapt VLMs for Polyp VQA. The goal is to improve the model’s ability to answer endoscopy-related visual questions while keeping fine-tuning feasible under limited resources. This is appropriate for the thesis setting because endoscopic VQA data is specialized and smaller than the generic multimodal corpora used to train foundation models.

The fine-tuning experiment is designed around two compact base models: MedGemma-4B-it and Qwen3.5-4B. MedGemma is a medical vision-language model based on the Gemma family and is designed to provide stronger medical image and text understanding than a general-purpose model of similar scale [40]. Qwen models provide a strong multilingual and instruction-following foundation, with Qwen3 extending the family toward hybrid reasoning and broader multilingual coverage [41]. Comparing these two backbones is useful because Polyp VQA requires both visual understanding and language generation: MedGemma contributes domain-specific medical pretraining, while Qwen provides a strong general instruction-following baseline.

LoRA also makes it practical to compare input formulations. In this work, Polyp VQA is evaluated with original-image input and segmentation-grounded input, allowing the analysis to separate the effect of model adaptation from the effect of spatial grounding. The segmentation-grounded setting uses the mask as a prior that indicates where the model should inspect the original image, rather than as a substitute for visual interpretation.

## 2.7 Polyp Segmentation and Neoplasm Characterization

Colonoscopy polyp segmentation is a key medical image analysis task because accurate localization of polyps can support colorectal cancer screening and clinical interpretation. The task can be binary, where the model separates polyp from background, or multi-class, where the

model also distinguishes neoplastic and non-neoplastic regions. Compared with general semantic segmentation, polyp segmentation is difficult because polyps vary greatly in size, color, texture, shape, and boundary clarity. The boundary between polyp tissue and surrounding mucosa can be ambiguous, and artifacts such as blur, specular highlights, low contrast, and imperfect bowel preparation can further reduce visual quality [32].

Public datasets have strongly shaped research in this area. Kvasir-SEG provides manually annotated gastrointestinal polyp images with segmentation masks and is widely used for benchmarking binary polyp segmentation [33]. For neoplasm characterization, the BKAI-IGH NeoPolyp dataset extends the problem from localization to multi-class segmentation, where the model must distinguish neoplastic and non-neoplastic regions. This makes the task closer to clinical interpretation than a pure foreground-background mask, but it also increases class imbalance and label ambiguity.

Early deep learning approaches for biomedical segmentation were strongly influenced by U-Net, whose encoder-decoder structure combines contextual feature extraction with precise localization through skip connections [29]. U-Net++ further redesigned skip pathways with nested dense connections to reduce the semantic gap between encoder and decoder features [30]. These architectures remain important baselines because medical datasets are often small and require models that can learn from limited annotated data.

DeepLabV3+ introduced another influential direction by combining atrous convolution, Atrous Spatial Pyramid Pooling, and a decoder for boundary refinement [31]. The multi-scale receptive fields are useful for polyp segmentation because lesions can occupy very different image areas. This thesis follows the DeepLabV3+ family for the deployed segmentation model and produces a three-class output. In the BKAI-IGH NeoPolyp-Small setting, DeepLabV3+ with RegNetx320 achieves the strongest Dice Score after post-processing among the evaluated models.

Attention-based and transformer-based methods address different limitations. PraNet uses a parallel partial decoder and reverse attention to refine boundary cues and recover missed regions [32]. Polyp-PVT replaces a CNN backbone with a pyramid vision transformer and introduces modules for cascaded fusion, camouflage identification, and similarity aggregation, improving robustness in challenging cases such as small objects and appearance variation [34]. More recent medical segmentation work also emphasizes efficient decoders; EMCAD proposes a multi-scale convolutional attention decoder that reduces parameters and FLOPs while maintaining strong performance across medical segmentation tasks [35].

Foundation segmentation models introduce another research direction. Polyp-SAM adapts the Segment Anything Model to the polyp domain and shows that foundation models can be transferred to medical segmentation, but domain adaptation remains necessary because generic segmentation priors do not automatically solve endoscopic boundary ambiguity [36]. This supports the broader design principle of the thesis: general foundation models are useful starting points, but task-specific adaptation and evaluation are still required for medical workflows.

In this thesis, segmentation is not only an end result but also an intermediate artifact for multimodal reasoning. A clean mask can help the Polyp VQA model focus on clinically relevant regions, while an incorrect mask can misdirect the answer. Therefore, segmentation quality, visual grounding, and VQA quality must be evaluated together rather than as isolated components.

## 2.8 Long-Term Memory for AI Agents

Most dialogue agents maintain short-term state within a session, but medical interaction often requires continuity across sessions. Long-term memory allows an agent to remember clin-

ically useful facts such as allergies, chronic diseases, medication use, repeated symptoms, and patient preferences.

In the proposed system, Mem0 is used as a patient-scoped long-term memory layer backed by vector retrieval. It does not replace LangGraph, RAG, the Knowledge Graph, or patient data storage. Instead, it provides additional context that helps agents personalize responses, support routing, and avoid repeatedly asking for information that was already provided.

Medical memory must be handled conservatively. A stored fact may represent a user-reported allergy or symptom history, but it should not be treated as a verified diagnosis. Memory is therefore used as supporting context rather than authoritative clinical truth, and it should remain subordinate to the current user input, retrieved medical evidence, and current image findings.

## 2.9 Human-in-the-loop for Medical AI

Human-in-the-loop validation is important in medical AI because user feedback can reveal unsafe, incomplete, or unclear responses. In this system, validation allows users to confirm or comment on system responses. This feedback is logged and can support quality review, memory selection, and future improvement.

# 3 Chapter 3. Proposed Multi-Agent Medical Assistant Architecture

## 3.1 Overall Architecture

The proposed system is organized as a multi-agent medical assistant with four main layers:

- **Decision and orchestration layer:** selects the appropriate processing path;
- **Text Module:** handles medical QA, multi-hop consultation, RAG, Knowledge Graph, and fallback search;
- **Vision Module:** handles general medical image interpretation, polyp segmentation, and Polyp VQA;
- **Memory and validation layer:** provides long-term patient context and human-in-the-loop feedback.

This architecture is designed to reduce reliance on a single model. Instead of asking one LLM to solve every task, the system delegates work to agents that are specialized for retrieval, graph reasoning, visual interpretation, segmentation, or VQA.

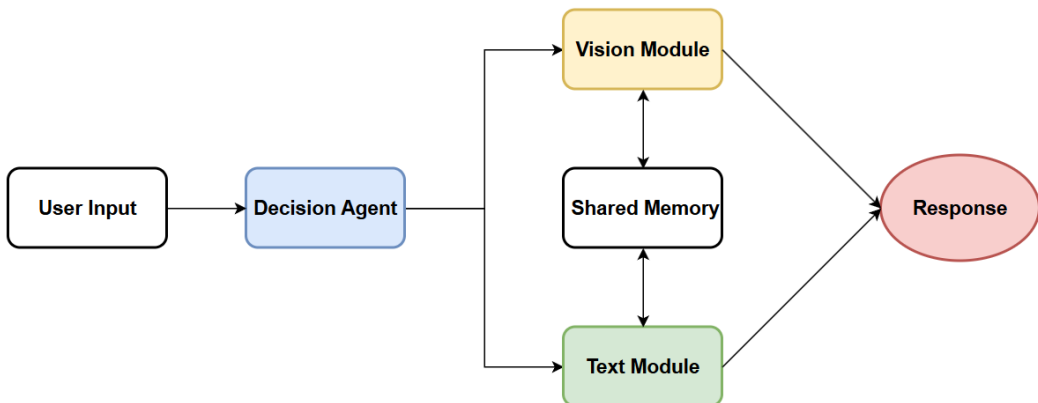


Figure 1: Overall architecture of the Multi-Agent Medical Assistant

Figure 1 shows that the system is not organized as a linear chatbot pipeline. The Decision Agent first interprets the modality and intent of the user input, then routes the request to the Text Module or Vision Module. Shared memory is placed between these modules rather than attached to only one agent. This design allows text-based reasoning and image-based reasoning to exchange context while still keeping their internal models and evidence sources separate.

The high-level architecture in Figure 1 establishes the main separation of responsibilities. The more detailed workflow in Figure 2 explains how these responsibilities are coordinated during execution and is used below to analyze the system benefits.

### 3.2 LangGraph-based Agent Orchestration

LangGraph is used as the workflow orchestration framework. It maintains state across the processing flow and connects decision nodes with task-specific agents. The graph-based structure is useful because medical consultation is not a simple single-step process. A query may require routing, follow-up questioning, retrieval, fallback search, image analysis, validation, or memory update.

The short-term state includes the current query, image path when available, selected agent, conversation history, patient identifier, and intermediate outputs such as segmentation paths or retrieved contexts. This state supports coherent routing and response generation within the current session.

The graph structure also makes the workflow easier to control than a purely prompt-based pipeline. Each node has a clear role: routing, retrieval, image analysis, response generation, validation, or memory update. This is useful in a medical context because the system can inspect intermediate outputs before producing the final answer. For example, if retrieval fails or the image route is uncertain, the workflow can move to a fallback path instead of forcing a response from incomplete evidence.

Another advantage is that LangGraph separates short-term state from long-term memory. Short-term state is useful for the current interaction, while Mem0 stores facts that may remain useful across sessions. This separation prevents temporary reasoning artifacts, such as a retrieved passage or a provisional model output, from automatically becoming persistent patient memory.

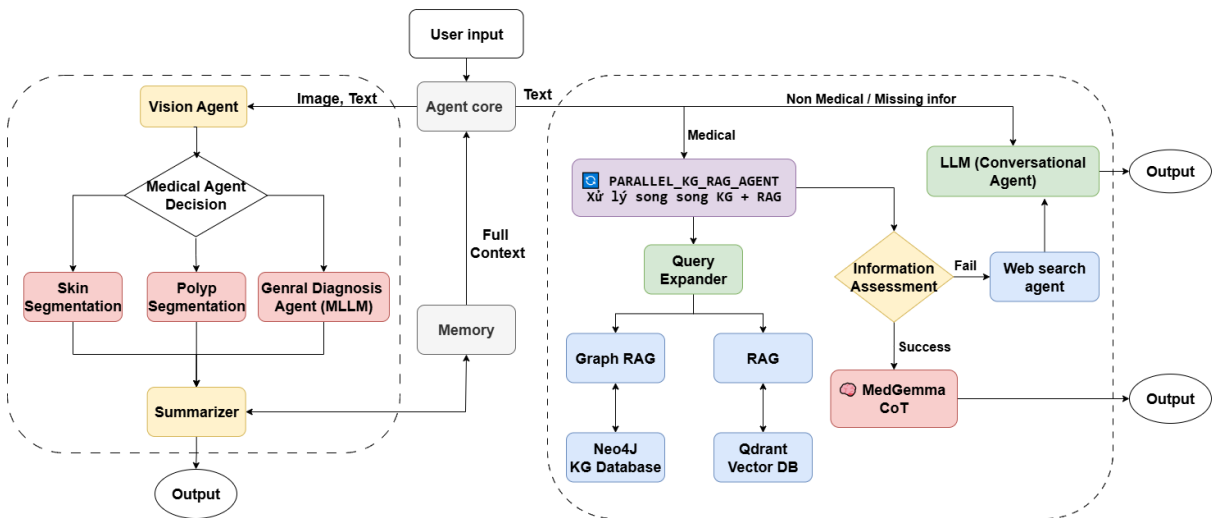


Figure 2: Detailed agent workflow and module coordination

Figure 2 shows both the system components and the routes between them. Table 1 summarizes the architectural benefits that follow from this workflow.

| Benefit                             | How Figure 2 supports the benefit                                                                                                                                                                                                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Functional decomposition</b>     | User input is routed from the Agent Core to specialized text and vision branches. This means heterogeneous medical tasks can be handled by agents with suitable tools instead of forcing one model to perform retrieval, reasoning, segmentation, and VQA at the same time.                |
| <b>Evidence specialization</b>      | KG, RAG, Web Search, shared memory, and segmentation masks appear as separate evidence channels. The system can therefore combine structured relations, explanatory documents, updated web evidence, patient context, and visual grounding without collapsing them into one opaque prompt. |
| <b>Graceful fallback</b>            | MedGemma CoT evaluates the selected KG/RAG context and routes weak-evidence cases to Web Search or conversational handling. This helps avoid premature answers when internal evidence is missing, uncertain, or clinically underspecified.                                                 |
| <b>Cross-modal continuity</b>       | Vision outputs and text reasoning interact through shared memory and workflow state. A user can upload an image, receive an initial interpretation, and then continue asking follow-up questions that reuse the image-derived context.                                                     |
| <b>Observability and evaluation</b> | Intermediate modules such as routing, query expansion, KG/RAG retrieval, segmentation, and response generation are explicit. Errors can therefore be traced to a specific stage, and the system can be evaluated beyond the final generated answer.                                        |

Table 1: Benefits of the multi-agent design illustrated by the detailed workflow

The table clarifies that the system is not merely a collection of independent models. Its value comes from how intermediate evidence is produced, assessed, reused, and routed before final response generation.

### 3.3 Agent Cooperation and Cross-module Workflows

A key characteristic of the proposed system is that agents do not operate as isolated modules. They cooperate through three shared objects: **workflow state**, **long-term memory**, and **intermediate evidence**. This means that the output of one agent can become structured input for another agent.

The cooperation mechanism can be summarized as follows:

- **Text cooperation:** Conversation Agent, Query Expansion, KG Agent, RAG Agent, Web Search Agent, and MedGemma CoT cooperate to decide whether to ask, retrieve, reason, or fall back.
- **Vision cooperation:** Polyp Segmentation Agent produces a lesion mask that becomes visual grounding for the Polyp VQA Agent.
- **Cross-modal cooperation:** image-derived context can support later text follow-up questions, while text context can guide image interpretation.
- **Memory-supported cooperation:** patient-reported facts can help routing and personalization, but they remain supporting context rather than confirmed clinical evidence.



| Cooperation pattern            | Agents involved                                                  | Shared evidence                                                                  | Main benefit               |
|--------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------|----------------------------|
| <b>Text reasoning</b>          | Conversation, Query Expansion, KG, RAG, MedGemma CoT, Web Search | Follow-up needs, expanded query, KG facts, retrieved passages, fallback evidence | Safer medical QA           |
| <b>Segmentation-to-VQA</b>     | Polyp Segmentation Agent and Polyp VQA Agent                     | Segmentation mask and overlay image                                              | Better visual grounding    |
| <b>Cross-modal follow-up</b>   | Vision agents, Conversation Agent, Text Module                   | Image result, short-term state, user follow-up question                          | Interactive consultation   |
| <b>Memory-assisted routing</b> | Memory Layer, Decision Agent, task-specific agents               | Patient-reported allergies, medication use, chronic conditions, prior findings   | Continuity across sessions |

Table 2: Main cooperation patterns in the proposed multi-agent architecture

**Text reasoning pattern.** The Text Module uses cooperation to avoid premature answer generation. The Conversation Agent first checks whether the user has provided enough clinical context. If the query is sufficiently specified, Query Expansion reformulates it into medically useful terms. The KG Agent then returns structured relations such as symptom–disease links, while the RAG Agent retrieves explanatory documents. These evidence sources are passed into MedGemma CoT, which evaluates information sufficiency and decides whether to synthesize an answer, ask a follow-up question, or activate fallback search.

**Segmentation-to-VQA pattern.** In the Vision Module, the Polyp Segmentation Agent produces a mask that localizes clinically relevant regions in a colonoscopy image. This mask is not only a final segmentation output; it is reused as a visual grounding signal for the Polyp VQA Agent. The VQA Agent can then answer natural-language questions using both the original image and the segmentation overlay.

**Cross-modal follow-up pattern.** After a user uploads a medical image, the interaction can continue as a dialogue. The user may ask what a highlighted region means, whether a finding is concerning, or what symptoms should be monitored. The system keeps image-derived context in short-term workflow state and routes later text questions back to the relevant visual evidence or to the Text Module for medical explanation.

**Memory-supported pattern.** Shared memory provides continuity across sessions. For example, previous patient-reported facts can help the Decision Agent route a new medication question more carefully. However, memory is not treated as a diagnosis. It supports context construction, while current symptoms, retrieved evidence, and image findings remain the primary basis for reasoning.

Overall, agent cooperation turns modularity into an evidence-passing workflow. Segmentation contributes spatial precision, VQA provides language-based interpretation, KG contributes structured relations, RAG provides explanatory grounding, and the Conversation Agent manages missing information. The architecture is therefore valuable because specialized agents produce intermediate evidence that other agents can inspect, reuse, and evaluate.

### 3.4 Decision Agent

The Decision Agent is responsible for determining which agent should handle a request. For text-only queries, it decides whether the query should be handled by Conversation Agent, KG/RAG reasoning, or Web Search. For image queries, it routes to General Medical Image Agent, Polyp Segmentation Agent, or Polyp VQA Agent.

The decision process considers four main signals:

- **Modality:** whether the input is text-only, image-only, or image with a question;
- **Intent:** whether the user asks for explanation, symptom consultation, segmentation, VQA, or fallback search;
- **Conversation context:** whether the current query refers to previous turns or uploaded images;
- **Patient context:** whether long-term memory provides relevant allergies, medications, chronic conditions, or prior findings.

| Routing case                           | Preferred route                           | Reason for this route                                                              |
|----------------------------------------|-------------------------------------------|------------------------------------------------------------------------------------|
| General or incomplete conversation     | Conversation Agent                        | Collect missing details or answer non-specialized dialogue safely.                 |
| Medical text query with enough context | KG/RAG reasoning and MedGemma CoT         | Retrieve evidence, compare sources, and synthesize a cautious medical response.    |
| Weak internal evidence                 | Web Search fallback or follow-up question | Avoid forcing an answer when internal sources do not support the query.            |
| General medical image                  | General Medical Image Agent               | Provide cautious visual interpretation without assuming a polyp-specific workflow. |
| Colonoscopy segmentation request       | Polyp Segmentation Agent                  | Produce a spatial mask for direct inspection and downstream grounding.             |
| Polyp image question                   | Polyp VQA Agent                           | Answer an image-question pair using the specialized VQA pipeline.                  |

Table 3: Decision Agent routing behavior

Routing errors are important because they can create downstream failures. If a symptom consultation is routed directly to answer generation, the system may skip necessary follow-up questions. If a general image is routed to the specialized polyp workflow, the segmentation model may produce meaningless output. If a polyp VQA question is routed to a generic VLM, the answer may lack domain-specific reasoning. Therefore, the Decision Agent is evaluated not only as a convenience component but also as a safety-critical part of the architecture.

**Conservative routing principle.** When the task is ambiguous, the system should prefer clarification or a safer general path over a highly specialized agent. Specialized agents are useful only when the intent and input type match their assumptions.

### 3.5 Knowledge and Context Sources

The system combines several knowledge and context sources. They are not equally reliable, and they should not be merged blindly. Table 4 summarizes the role and limitation of each source.

| Source                  | Primary contribution                                             | Main limitation                                                                    |
|-------------------------|------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>RAG documents</b>    | Vietnamese explanatory medical text stored in a vector database. | May retrieve text that is semantically similar but clinically weak or too general. |
| <b>Knowledge Graph</b>  | Structured disease, symptom, drug, and relation evidence.        | May be incomplete or too sparse for patient-friendly explanation.                  |
| <b>Web Search</b>       | Fallback coverage for missing or updated information.            | Requires careful source filtering and more conservative claims.                    |
| <b>Short-term state</b> | Current conversation context and intermediate workflow outputs.  | Useful only within the current session and may include provisional reasoning.      |
| <b>Long-term memory</b> | Patient-reported facts retrieved through Mem0.                   | May be outdated, self-reported, or incomplete.                                     |
| <b>Visual masks</b>     | Spatial grounding for Polyp VQA.                                 | Depends on segmentation quality and may propagate mask errors.                     |

Table 4: Knowledge and context sources with reliability considerations

This combination allows the system to support both explanatory medical answers and structured reasoning. The final response should reflect the strength and uncertainty of each source rather than treating all retrieved context as equally authoritative.

### 3.6 Safety and Human-in-the-loop Validation

The system includes safety-oriented response behavior. It provides medical disclaimers, recommends professional consultation when necessary, and avoids presenting AI output as a definitive diagnosis. The human-in-the-loop validation mechanism allows users to confirm or comment on generated responses. This feedback can support later quality review and can help determine which facts are reliable enough to be stored in long-term memory.

- **Before generation:** routing, retrieval, and sufficiency checks reduce unsupported answers.
- **During generation:** prompts require uncertainty, warning signs, and care guidance.
- **After generation:** human validation collects user feedback for review and memory governance.

Human validation is placed after the AI response rather than inside every internal reasoning step. This keeps the system usable while still allowing feedback to be collected. In the scope of this thesis, validation is treated as a safety and review mechanism, not as proof of clinical correctness. The system remains a decision-support tool and should not be interpreted as replacing professional medical judgment.

## 4 Chapter 4. Text Module: Retrieval and Reasoning

### 4.1 Role and Processing Flow of the Text Module

The Text Module is responsible for medical reasoning over user-provided language. It handles questions about symptoms, diseases, drugs, medical concepts, and follow-up concerns. Its main goal is to produce responses that are grounded, cautious, and aware of missing clinical information.

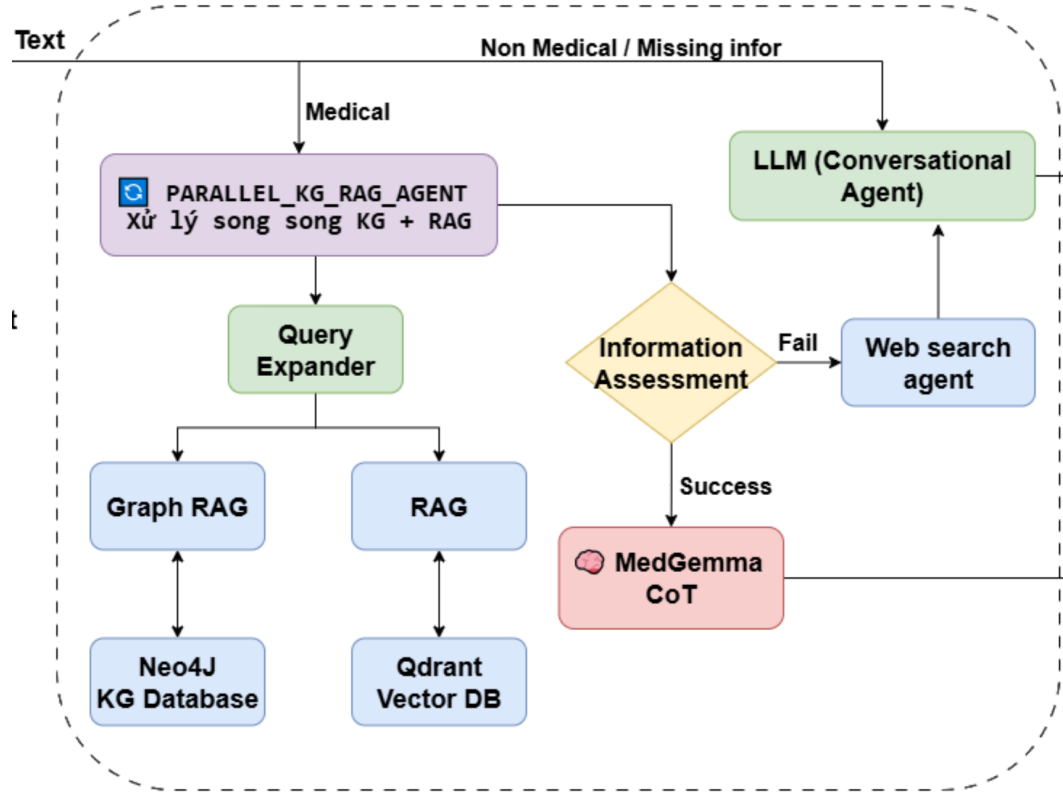


Figure 3: Text Module architecture for medical knowledge retrieval

Figure 3 provides the high-level view of the Text Module. It shows that text reasoning begins with user input classification, then branches into conversational handling, KG/RAG evidence retrieval, MedGemma CoT assessment, fallback search, and final response generation. Although the workflow diagram may visualize assessment as a separate decision node, this node should be interpreted as a logical step inside MedGemma CoT rather than as an independent agent. This figure is therefore introduced at the beginning of the chapter before the individual agents are discussed in detail.

**Core idea.** The Text Module is not only a retrieval component. It is a control layer that decides when to ask, when to retrieve, when to synthesize evidence, and when to avoid a patient-specific conclusion.

The general processing flow begins with input analysis, then moves to clarification when the query is clinically underspecified, evidence retrieval when the query is answerable from medical sources, MedGemma CoT assessment when candidate evidence is available, and final response generation only after the system has enough context to communicate safely. This flow is designed to make the Text Module cautious by default. The module does not assume that every query should immediately trigger a final answer. Instead, it separates the tasks of understanding the user intent, checking information completeness, retrieving evidence, and generating the final response.

The Text Module therefore acts as the main reasoning layer for non-image medical queries. It must handle two very different types of user input. The first type is educational, such as asking what a disease is or how a medication works. The second type is consultation-oriented, such as describing symptoms and asking what might be happening. The second type requires stricter information sufficiency checks because the same symptom can have different meanings depending on duration, severity, age, medical history, and associated signs.

The workflow in Figure 2 also shows that text reasoning is not reduced to document retrieval alone. The Text Module combines conversational intake, KG/RAG context selection,

MedGemma CoT reasoning, web fallback, and final medical response generation. This layered structure is important because the main risk in medical QA is not only whether the system can retrieve facts, but whether the reasoning component knows when the available context is insufficient for a patient-specific conclusion.

## 4.2 Text Query Types and Routing Behavior

The Text Module is designed around the observation that medical text queries are not homogeneous. Different query types require different agent behavior. A general educational question can usually be answered after retrieval, while a patient-specific symptom question may require follow-up questioning before retrieval. A medication-related question may require checking contraindications, allergy context, and source reliability. A non-medical or socially conversational input should not be forced into a medical reasoning pipeline.

| Query type                              | Main agent behavior                                                                                         | Reasoning requirement                                                                                   |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Medical education</b>                | Use RAG and/or KG to retrieve reliable medical knowledge.                                                   | Provide grounded explanation without over-personalizing the answer.                                     |
| <b>Symptom consultation</b>             | Pass user context to MedGemma CoT for sufficiency assessment and ask follow-up questions if needed.         | Avoid premature diagnosis when onset, severity, duration, risk factors, or warning signs are missing.   |
| <b>Structured relation query</b>        | Route to KG reasoning and use RAG for explanation when needed.                                              | Connect entities such as symptoms, diseases, drugs, body parts, and risk factors.                       |
| <b>Medication or treatment question</b> | Use retrieval cautiously and consider patient context such as allergies or chronic diseases when available. | Avoid unsafe recommendations and encourage professional consultation for dosage or treatment decisions. |
| <b>Missing or non-medical input</b>     | Use the conversational agent or request clarification.                                                      | Prevent irrelevant retrieval and keep the system aligned with user intent.                              |

Table 5: Text query types and corresponding routing behavior

Table 5 shows why routing is part of the reasoning problem. If every text query is treated as a retrieval query, the system may return medical documents even when the user actually needs clarification. Conversely, if every symptom query is treated as a conversational query, the system may underuse available medical evidence. The Text Module therefore uses routing to decide when to ask, when to retrieve, when to reason over graph relations, and when to synthesize a response.

## 4.3 Conversation Agent and Multi-hop QA

The Conversation Agent handles cases where the user’s query is incomplete or conversational. In medical consultation, missing information is common. For example, a symptom description may lack onset time, severity, location, duration, associated symptoms, medical history, or current medication use.

Multi-hop QA is used to collect this missing information step by step. Instead of generating a diagnostic suggestion immediately, the agent asks focused follow-up questions. This process

helps the system approximate a structured clinical intake conversation. The goal is not to replace a doctor, but to reduce the risk of giving an answer from insufficient evidence.

The follow-up questions are designed to be concise and clinically relevant. For symptom-based consultation, the system prioritizes red flags and high-impact missing variables. For example, in a fever-related query, duration, peak temperature, associated respiratory or gastrointestinal symptoms, medication use, and risk factors may change the response. In a pain-related query, location, severity, onset, radiation, and accompanying symptoms are important.

This behavior improves usability because the system does not overwhelm the user with a full medical form. It asks only the next useful question based on the current context.

The multi-hop design also improves traceability. Each follow-up question corresponds to a missing clinical variable, such as onset, duration, severity, location, associated symptoms, medication use, allergy history, or risk factors. This makes it possible to analyze whether the agent is asking useful questions or merely extending the conversation. In evaluation, this behavior is reflected by Multi-Hop Awareness Score, which rewards the system for recognizing when additional information is needed.

#### 4.3.1 Single-turn Limitation versus Multi-hop Clarification

The difference between a traditional single-turn response and the proposed multi-hop QA behavior is illustrated in Figures 4 and 5. These two examples use the same general symptom scenario, but the response strategies are different.

### Nhược điểm của phương pháp truyền thống

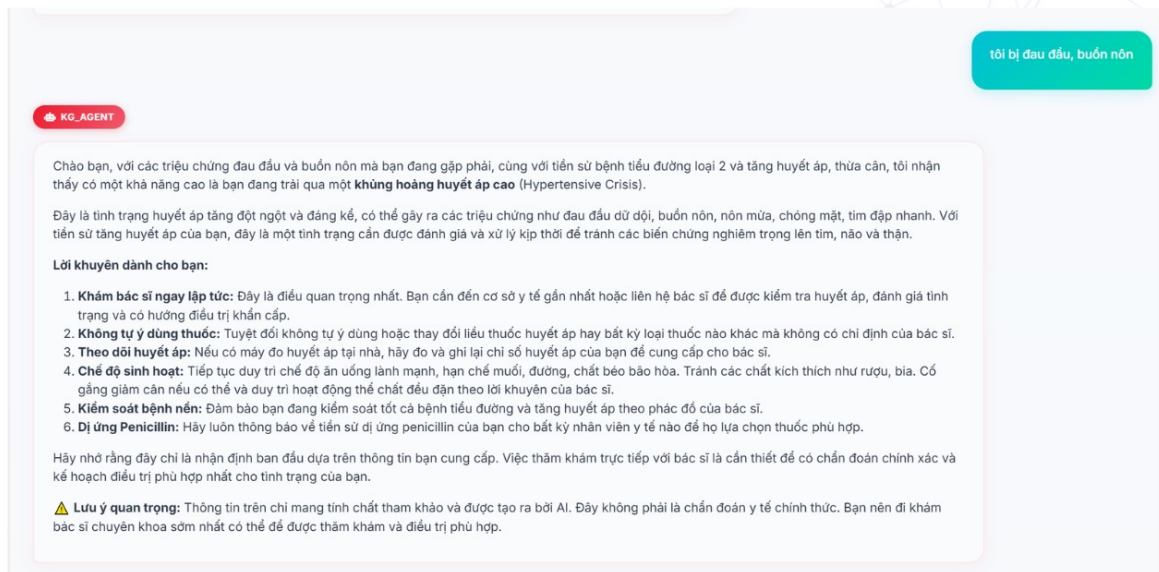


Figure 4: Traditional single-turn approach with limited clinical context

Figure 4 shows the main weakness of a traditional single-turn medical assistant. The user provides a short symptom description, such as headache and nausea, but the system immediately produces a relatively strong medical interpretation. Even if the response includes disclaimers, the reasoning is still based on an incomplete clinical picture. Important variables such as current blood pressure, onset time, severity, neurological symptoms, medication adherence, and warning signs are not yet available. As a result, the model may overcommit to one hypothesis or provide advice that is not sufficiently adapted to the actual situation.

This failure mode is particularly important in medical AI because symptoms are rarely specific by themselves. Headache and nausea can be associated with many conditions, ranging from mild fatigue or migraine to hypertensive urgency, infection, medication side effects, or neuro-

logical emergencies. A single-turn answer may sound helpful because it is detailed, but detail does not guarantee clinical appropriateness. The core problem is that the model is answering before it has enough discriminative information.

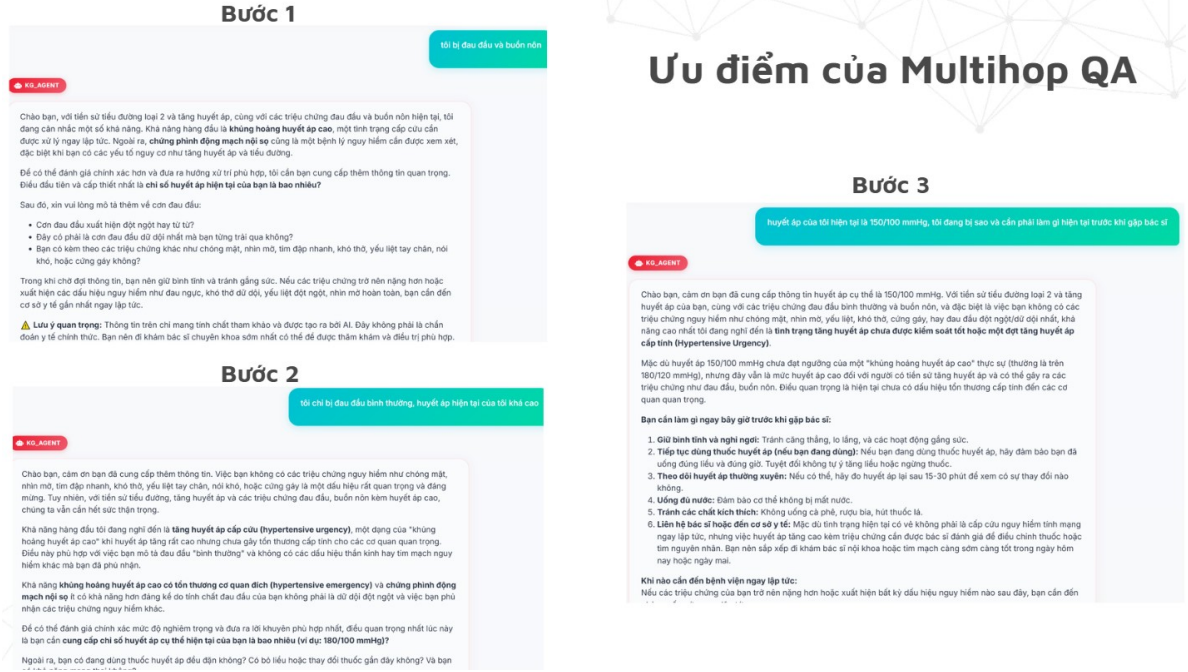


Figure 5: Multi-hop QA approach with iterative clarification and safer response refinement

Figure 5 demonstrates the advantage of the multi-hop QA workflow. Instead of producing a final answer immediately, the system first identifies missing clinical information and asks a focused follow-up question. When the user provides additional context, such as a blood pressure value or the absence of severe neurological symptoms, the system updates its interpretation. The answer then becomes more specific: it can distinguish between hypertensive emergency, hypertensive urgency, and a high but not immediately life-threatening blood pressure state.

This iterative process improves the response in three ways. First, it improves **clinical specificity** because the system can condition its answer on newly provided variables. Second, it improves **safety** because red flags are checked before giving ordinary advice. Third, it improves **explainability** because each follow-up question corresponds to a missing piece of information that affects the final decision. In this sense, multi-hop QA is not only a conversational feature; it is a safety mechanism for controlling when the system is allowed to answer.

The comparison also clarifies the role of the Conversation Agent. Its purpose is not to ask many questions for their own sake, but to ask the next question that most reduces clinical uncertainty. A good follow-up question should be short, relevant, and oriented toward high-impact variables such as red flags, vital signs, duration, severity, medication use, and patient history. Once sufficient information is available, the system can move from clarification to evidence retrieval and response generation.

#### 4.3.2 Illustrative Multi-hop Consultation Flow

An illustrative symptom consultation demonstrates why multi-hop behavior is needed. Suppose the user writes: "I have stomach pain and nausea. What disease could it be?" A single-turn chatbot may immediately list gastritis, food poisoning, reflux, or appendicitis. However, this answer is clinically weak because the initial input lacks duration, pain location, severity, fever, vomiting, stool changes, medication use, pregnancy status, and warning signs.

In the proposed workflow, the Conversation Agent first treats the query as underspecified.



Instead of giving a strong conclusion, it asks a focused follow-up question such as: “Where is the pain located, how long has it lasted, and do you have fever, vomiting blood, black stool, severe pain, or persistent vomiting?” This question is not arbitrary; it targets variables that can change the safety framing of the response. If the user later reports mild epigastric pain after meals without red flags, the system can retrieve information about gastritis or reflux and provide cautious advice. If the user reports severe right-lower abdominal pain with fever, the system should prioritize urgent professional care rather than ordinary health education.

This example illustrates the difference between answer generation and consultation support. The goal of the Conversation Agent is not to prolong the dialogue, but to determine whether the current evidence is sufficient for the next safe action. The final response may still include possible causes, but it should clearly separate likely explanations, missing information, warning signs, and next steps.

## 4.4 Clinical Context Requirements before MedGemma CoT

Information sufficiency is not implemented as a standalone agent in the proposed workflow. It is a decision step inside MedGemma CoT. Before that step can operate effectively, the upstream Text Module must provide two kinds of input: the user’s clinical context and high-quality evidence retrieved by KG/RAG agents.

This separation of responsibility is important. KG and RAG agents are responsible for selecting useful context data: structured graph relations, candidate diseases, matched symptoms, and explanatory medical passages. MedGemma CoT is then responsible for deciding whether that context is sufficient for a safe response, whether red flags are present, and whether the system should answer, ask a follow-up question, or trigger fallback search.

**Sufficiency is a decision boundary.** A query can have retrieved evidence and still be clinically insufficient. The system should not confuse “documents were found” with “the user gave enough context for patient-specific advice.”

Many medical questions should not be answered directly. A safe assistant should distinguish between:

- general medical education questions that can be answered from knowledge sources;
- symptom-based questions that require follow-up details;
- high-risk cases that should recommend immediate professional care;
- out-of-scope or non-medical queries.

Information sufficiency is also connected to evaluation. A model that refuses to conclude when the evidence is incomplete may appear less decisive, but it is safer in medical reasoning. Therefore, this thesis uses Multi-Hop Awareness Score to measure whether MedGemma CoT can recognize insufficient information rather than forcing an answer.

In implementation, information sufficiency should be interpreted as a dynamic threshold rather than a fixed checklist. The threshold is lower for general education and higher for patient-specific recommendations, medication-related questions, and symptoms associated with potentially urgent conditions. This design allows the Text Module to answer low-risk factual questions while remaining cautious when the user appears to request personalized medical judgment.

An important insight is that information sufficiency is an architectural control point inside MedGemma CoT, not only a prompt instruction. The result of this assessment changes the route of execution: the system may continue to response synthesis, ask the Conversation Agent to collect missing details, or invoke Web Search when KG/RAG context is weak.

This step prevents a common failure mode of retrieval-augmented systems. A retrieved passage can make a response appear grounded even when the original user question is incomplete.



For example, if the user says only “I have chest pain,” RAG may retrieve documents about angina, reflux, anxiety, or muscle pain. These documents are medically relevant, but they do not solve the missing-context problem. The system must first identify urgent warning signs and ask about onset, severity, radiation, shortness of breath, sweating, and cardiovascular history. Therefore, evidence availability and clinical sufficiency are related but not identical.

## 4.5 Medical Knowledge Retrieval with RAG and Knowledge Graph

The system uses both RAG and Knowledge Graph retrieval because they provide complementary strengths.

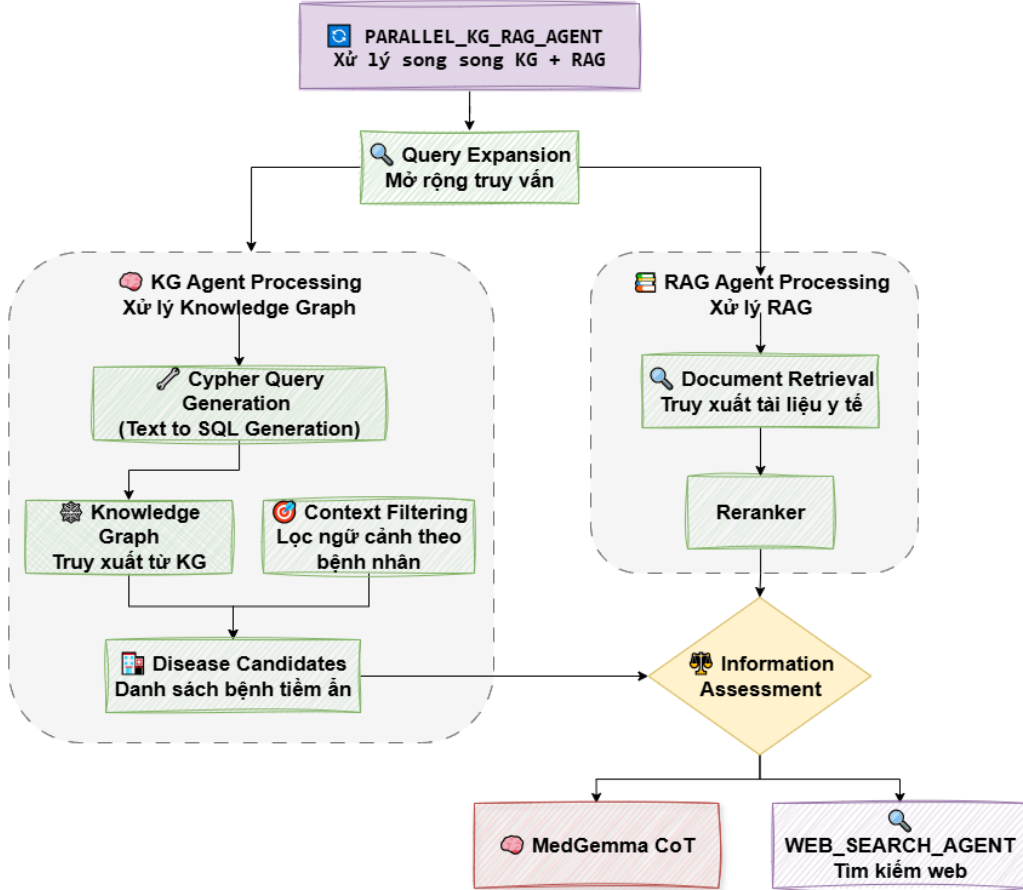


Figure 6: Parallel KG-RAG processing in the Text Module

The parallel design in Figure 6 reflects a central assumption of this thesis: unstructured and structured medical knowledge are complementary. RAG is strong when the system needs explanatory evidence, while KG reasoning is strong when the question depends on explicit relations between entities such as symptoms, diseases, drugs, and body parts. Running these branches in parallel allows the system to compare different evidence forms before generating an answer.

### 4.5.1 Data Retrieval Layer

The data retrieval layer is responsible for collecting candidate evidence before medical reasoning begins. In the proposed Text Module, retrieval is not limited to one database. The RAG branch retrieves unstructured medical documents from the Qdrant vector database, while the KG branch retrieves structured disease and symptom relations from Neo4j. This dual-source design allows the system to use both semantic similarity and explicit medical relations.

Before retrieval, Query Expansion reformulates the user question by adding related medical

terms, synonyms, anatomical terms, and possible disease names. This is especially important for Vietnamese medical queries because users may describe symptoms in informal language, while the data source may use more formal medical terminology. For example, a user may say “đau bao tử”, while relevant documents may use terms such as epigastric pain, gastritis, reflux, or stomach pain.

The vector retrieval branch uses dense embedding search. In the implementation, documents are embedded with `google/embeddinggemma-300m` and stored in Qdrant using cosine similarity. Dense retrieval is useful because it can match semantically related expressions even when the exact keywords differ. The retrieval design can also be extended with hybrid search, where dense vector search is combined with keyword-based retrieval such as BM25. This is useful in medical QA because some terms, drug names, disease names, and abbreviations require exact lexical matching, while symptom descriptions often benefit from semantic matching.

After candidate documents are retrieved, a reranker reorders them according to query-document relevance. This step is important because initial vector similarity may return documents that are semantically close but clinically less useful. The reranker helps prioritize documents that better match the actual user question before MedGemma CoT receives the selected context.

**Retrieval interpretation.** RAG is most useful for explanatory medical text, while Knowledge Graph retrieval is most useful for structured relationships. The system uses both because medical QA often needs a relation-level hypothesis and a patient-friendly explanation.

#### 4.5.2 KG Agent

The KG Agent retrieves structured medical knowledge from Neo4j. Its main role is to convert natural-language medical questions into graph queries and then use the retrieved relations as structured evidence. This is different from RAG because the KG branch does not only search for similar text; it traverses explicit relations between entities such as diseases, symptoms, treatments, medications, advice, and patient-related attributes.

The Cypher Query Service is responsible for translating user questions into Cypher queries. In the implementation, this process is supported by `GraphCypherQChain` and few-shot examples. Few-shot prompting is useful because medical graph queries often require a stable query pattern: identify the target entity, traverse relevant relations, and return disease candidates or associated attributes. A direct LLM answer may be fluent but unsupported, while a Cypher query forces the reasoning step to retrieve concrete graph evidence.

The KG Agent also supports patient-specific context integration. Retrieved graph results can be filtered using patient information such as age, gender, medical history, current diseases, and symptom context. This prevents the system from returning graph results that are structurally related but clinically less relevant to the patient. For example, a disease relation may exist in the graph, but it may be less appropriate if it mainly concerns a different age group or conflicts with the patient’s reported history.

Context filtering can use embedding similarity between the patient context and disease descriptions. The filtered results are then ranked as disease candidates, including information such as matched symptoms, total symptoms, and similarity scores. This makes the KG output more than a raw list of relations: it becomes a patient-aware candidate set that can be inspected by MedGemma CoT before response generation.

#### 4.5.3 RAG Agent

The RAG Agent retrieves and processes unstructured medical documents, mainly from VimedAQA through Qdrant. Its purpose is to provide explanatory medical context that can be

used in patient-facing responses. While the KG Agent is strong at structured relation reasoning, the RAG Agent is better suited for definitions, disease descriptions, general advice, medication explanations, and practical health information.

The RAG branch follows three main steps. First, Query Expansion improves the input query by adding relevant medical terms and synonyms. Second, dense retrieval searches the vector database for top- $k$  semantically related passages. Third, the reranker sorts the retrieved passages by practical relevance to the user query. This reranking step helps reduce noise before the response generator synthesizes an answer.

RAG also improves transparency because the final answer can be grounded in retrieved documents rather than only in the parametric memory of the LLM. However, retrieved documents should still be assessed critically. A passage can be medically valid but irrelevant to the patient’s specific situation. Therefore, the RAG output is passed as selected context into MedGemma CoT instead of being used blindly.

The KG and RAG branches therefore serve different but complementary purposes. KG contributes compact relational structure and candidate hypotheses, while RAG contributes explanatory context and patient-facing language. This hybrid design is evaluated in the Text Module benchmark in Table 13, where KG + MedLLM achieves the strongest result on both One-Hop Accuracy and Multi-Hop Awareness Score. RAG remains necessary for richer explanation, but the benchmark shows that structured KG evidence improves relation-oriented reasoning. The contribution of the Text Module is therefore not choosing KG or RAG in isolation, but coordinating them according to the evidence need of each query.

#### 4.5.4 Coordinated KG-RAG Evidence Construction

The KG and RAG branches contribute different forms of evidence. The KG branch is useful for identifying candidate relations, such as a disease being associated with a symptom or a drug being related to a disease category. The RAG branch is useful for converting those relations into explanatory language that can be understood by non-expert users. This distinction matters because a medical assistant must both reason and communicate.

For example, when the user asks about a symptom cluster, the KG branch can return candidate diseases connected to the symptoms. These candidates should not be treated as diagnoses. Instead, they serve as structured hypotheses that guide retrieval and response generation. The RAG branch can then retrieve documents explaining the candidate conditions, typical symptoms, warning signs, and general management. The response generator can synthesize these sources into a cautious answer: possible explanations, why they may be relevant, what information is missing, and when to seek medical care.

This coordination also reduces two opposite risks. If the system relies only on KG, the answer may become too sparse and relation-like, lacking patient-friendly explanation. If the system relies only on RAG, the answer may become verbose and less structured, especially when retrieved passages contain overlapping or weakly relevant information. By combining both, the Text Module can use KG for structure and RAG for explanation.

| Stage                     | Agent output                                                                                                 | Role in final response                                                                                        |
|---------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Query Expansion</b>    | Medical synonyms, related symptoms, disease names, and anatomical terms.                                     | Bridges colloquial user language and formal medical terminology.                                              |
| <b>KG Agent</b>           | Structured entity relations and candidate disease-symptom links.                                             | Provides compact reasoning structure and candidate hypotheses.                                                |
| <b>RAG Agent</b>          | Retrieved explanatory passages from medical documents.                                                       | Provides patient-friendly explanation and supporting context.                                                 |
| <b>MedGemma CoT</b>       | Candidate analysis, red-flag screening, and information sufficiency assessment over selected KG/RAG context. | Determines whether to answer, ask follow-up questions, or use fallback search.                                |
| <b>Response Generator</b> | Structured medical response with uncertainty and safety framing.                                             | Converts CoT-reviewed evidence into an understandable answer without presenting it as a definitive diagnosis. |

Table 6: Coordinated evidence construction in the Text Module

## 4.6 Parallel Reasoning and Fallback Strategy

The system can retrieve and filter RAG and Knowledge Graph results in parallel. These agents do not make the final medical decision; they select context for MedGemma CoT. If the selected internal context is insufficient after CoT assessment, the system can use Web Search as a fallback or ask the user for missing information.

This strategy reduces dependence on any single source. RAG may fail if the relevant document is not retrieved, while the Knowledge Graph may fail if the needed relation is missing. If one internal source is weak, the other source can still provide partial support. If both internal sources are weak, Web Search becomes a controlled fallback rather than an uncontrolled first response. Parallel retrieval does not mean that all sources are merged blindly: KG/RAG agents first select and filter context based on relevance, patient context, and retrieval quality, and MedGemma CoT then compares source consistency and clinical sufficiency.

## 4.7 MedGemma CoT for Medical Decision Support

After KG and RAG agents select candidate context, the MedGemma CoT component performs evidence synthesis and decision support. Web Search is treated as a fallback source when internal context is weak or incomplete, not as the default first source. The role of MedGemma CoT is not to create a definitive diagnosis, but to organize available evidence into a clinically cautious reasoning process. This component receives candidate diseases, matched symptoms, retrieved passages, patient context, and the current conversation state.

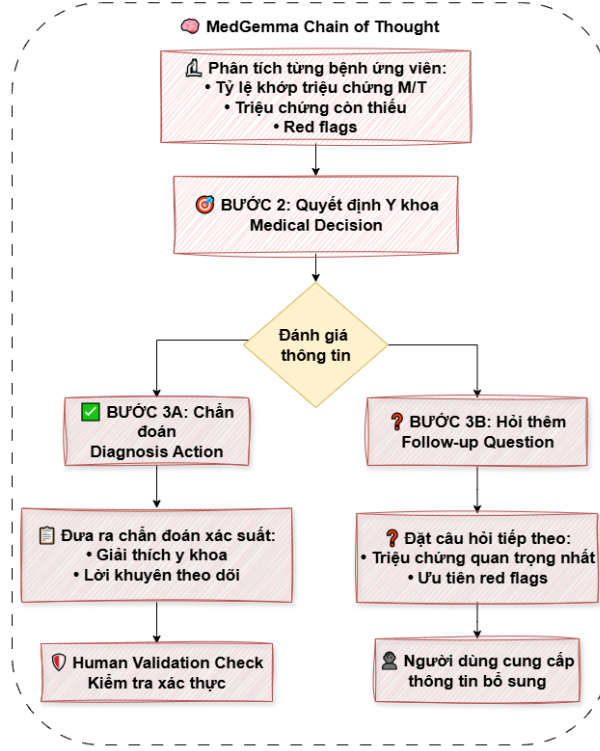


Figure 7: Medical chain-of-thought and information sufficiency flow

Figure 7 summarizes the decision flow of the MedGemma CoT component: it analyzes candidate diseases, checks red flags, evaluates information sufficiency, and then either produces a cautious diagnostic suggestion or asks follow-up questions.

The first step is candidate-level analysis. For each candidate condition, the model compares the user’s reported symptoms with the symptom profile retrieved from KG/RAG. This includes estimating whether the candidate explains the main complaint, which symptoms are matched, which symptoms are missing, and whether the available evidence is too weak. This step is important because a candidate disease should not be listed only because it appears in a retrieved document or graph relation.

The second step is red-flag screening. MedGemma CoT checks whether the conversation contains warning signs such as severe pain, difficulty breathing, neurological symptoms, bleeding, persistent vomiting, high fever, chest pain, or other urgent indicators depending on the symptom category. If red flags are present, the system should prioritize escalation and professional consultation instead of continuing ordinary explanation.

The third step is medical decision branching. If the retrieved evidence and user context are sufficient, the system can synthesize a cautious diagnostic suggestion with likely explanations and monitoring advice. If the evidence is insufficient, the system should activate multi-hop QA and ask the next clinically useful follow-up question. This branch is central to the safety design: the system is allowed to defer the answer when the information is incomplete.

| CoT step                      | Input considered                                                               | Output behavior                                                                                |
|-------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Candidate analysis</b>     | Disease candidates, matched symptoms, retrieved passages, and patient context. | Compare possible conditions and identify which candidates are plausible, weak, or unsupported. |
| <b>Red-flag screening</b>     | Reported symptoms, severity, duration, urgent signs, and risk factors.         | Escalate to urgent-care advice when dangerous patterns appear.                                 |
| <b>Information assessment</b> | Completeness of clinical variables and consistency of evidence sources.        | Decide whether to answer, ask follow-up questions, or use fall-back search.                    |
| <b>Response synthesis</b>     | Filtered evidence, candidate ranking, and safety constraints.                  | Produce a cautious explanation, possible causes, missing information, and next-step guidance.  |

Table 7: MedGemma CoT decision-support stages

This design connects retrieval with safe response generation. KG and RAG provide evidence, but MedGemma CoT decides how that evidence should be interpreted under uncertainty. A high-ranking candidate can still be rejected if key symptoms are missing, and a low-information query can trigger follow-up questioning even when retrieval returns many documents. Therefore, the MedGemma CoT stage prevents the system from confusing evidence retrieval with clinical sufficiency.

## 4.8 Safe Medical Response Generation

The final response is generated from the available evidence and conversation context. A safe medical response should:

- answer the user’s question directly when possible;
- explain the reasoning in understandable language;
- cite or mention sources when available;
- state uncertainty when information is incomplete;
- recommend professional care for severe or urgent symptoms;
- avoid presenting the AI output as a definitive diagnosis.

The generated response is then available for human-in-the-loop validation.

The final response is therefore the result of a controlled reasoning pipeline rather than a raw model completion. This design helps separate evidence collection from natural-language generation, making the system easier to analyze and improve. The response format should be adapted to risk level: low-risk educational questions can receive concise explanations; symptom-based questions should include known information, missing information, warning signs, and next steps; urgent or red-flag cases should prioritize escalation over detailed explanation.

The response generator should also preserve the provenance of reasoning as much as possible. If the answer is mainly based on RAG evidence, it should emphasize retrieved medical information. If the answer depends on KG relations, it should explain that the listed conditions are related possibilities rather than confirmed diagnoses. If Web Search is used, the response should be more conservative because source quality may vary. This provenance-aware response style improves user trust and makes the system easier to audit.

In addition, the response should separate *medical knowledge* from *patient-specific interpretation*. For example, the system can explain that gastritis commonly causes epigastric pain and

nausea, but it should not conclude that the user has gastritis without sufficient context. This separation is central to safe medical assistance: the assistant may provide useful education and triage guidance, but it should avoid replacing clinical diagnosis.

## **5 Chapter 5. Vision Module: General Medical Image Analysis, Polyp Segmentation, and Polyp VQA**

### **5.1 Role and Processing Flow of the Vision Module**

The Vision Module processes image-based medical inputs. It supports general medical image interpretation and a specialized colonoscopy polyp workflow. The module is designed to coordinate visual models with the multi-agent architecture so that image findings can be combined with text questions and patient context.

The module follows a task-specific design. General medical image interpretation is kept broad and cautious, while polyp-related processing is more specialized and more formally evaluated. This allows the system to support multiple image inputs without treating all image types as equally reliable.

The design intentionally separates general image analysis from specialized polyp analysis. A general VLM may be useful for describing visible patterns, but colonoscopy polyp segmentation and Polyp VQA require domain-specific assumptions, datasets, and evaluation metrics. This separation makes the system more transparent: the user and evaluator can identify whether an answer comes from a general visual model or from the specialized polyp workflow.

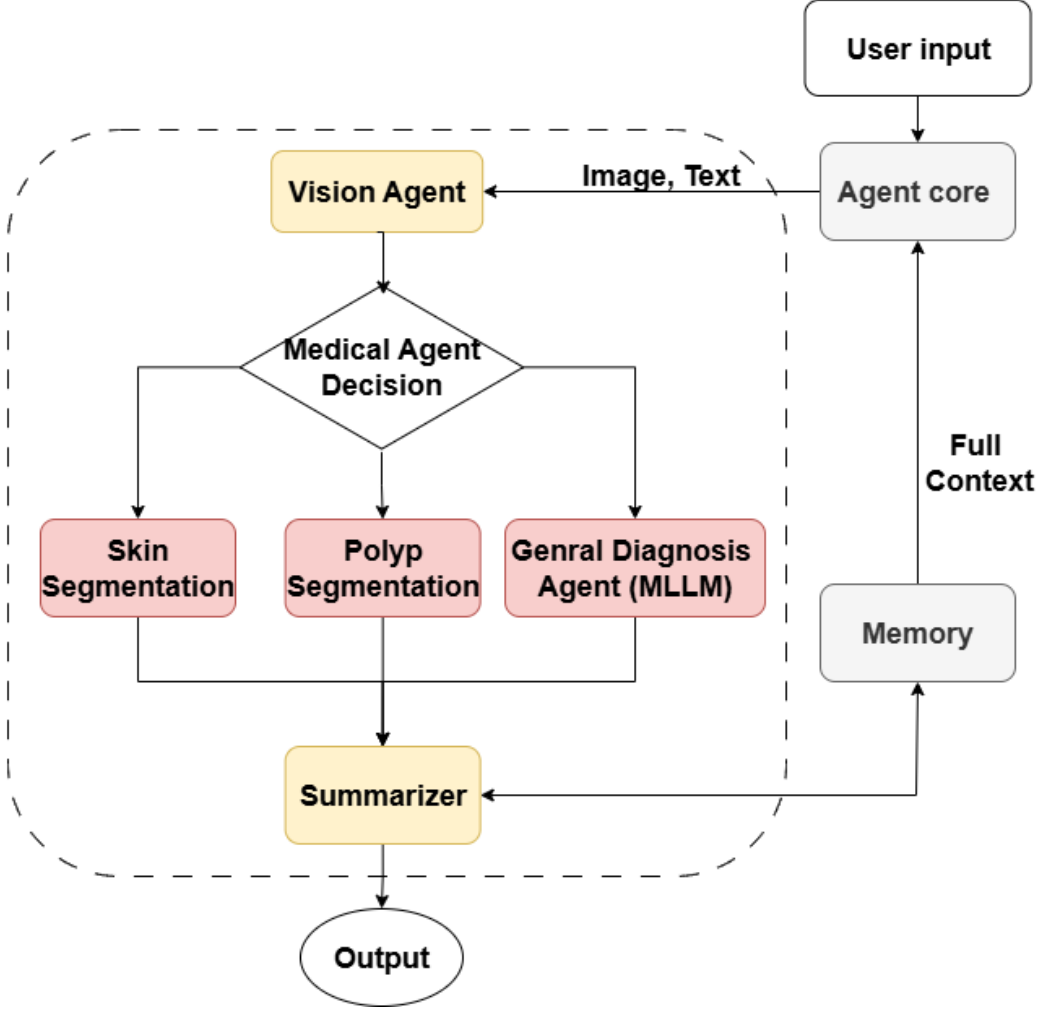


Figure 8: Vision Module workflow for medical image analysis

## 5.2 Image Routing and Visual Task Classification

When an image is uploaded, the system determines whether the task is general image interpretation, polyp segmentation, or Polyp VQA. If the user explicitly requests mask generation or segmentation, the image is routed to the Polyp Segmentation Agent. If the user asks a question about an endoscopic polyp image, the image is routed to the Polyp VQA Agent. Other medical images can be handled by the General Medical Image Agent.

This routing step is important because the same image can support different tasks. A user may ask for a segmentation mask, a description of visible findings, or a question about risk. Routing prevents the system from applying the wrong agent to the wrong task.

Visual routing also depends on the accompanying text. An image without a question may require general description, while the same image with a question about polyp type should trigger Polyp VQA. If the user asks specifically for localization or mask generation, segmentation becomes the primary task. The image and the text prompt are therefore interpreted together rather than independently.

## 5.3 General Medical Image Agent

The General Medical Image Agent handles medical images outside the specialized polyp workflow. It uses a multimodal model to describe visible findings, provide cautious interpretation, and recommend appropriate follow-up when necessary.

This agent is positioned as a support tool rather than a diagnostic authority. Its output



should be interpreted as preliminary information and should not replace clinical review by trained professionals.

Because the General Medical Image Agent covers a broad space of image modalities, it is not the main formal evaluation target in this thesis. Its role is to provide system coverage and demonstrate extensibility of the vision pipeline. The specialized polyp workflow receives more detailed quantitative evaluation.

The agent should avoid unsupported claims about diagnosis, severity, or treatment. Its most appropriate output style is a cautious description of visible content, possible medical relevance, and recommendation for professional interpretation when the image is clinically significant. This safety framing is necessary because different medical image modalities require different expert knowledge and acquisition conditions.

## 5.4 Polyp Segmentation Agent

The Polyp Segmentation Agent is designed for colonoscopy images. Its task is to segment polyp regions and distinguish between background, neoplastic polyp regions, and non-neoplastic polyp regions. The segmentation output is useful for two purposes: direct visual interpretation and visual grounding for Polyp VQA.

This agent converts an image-level input into a spatially localized representation. That representation is easier to inspect than a text-only answer and can be passed to other agents. In the proposed architecture, segmentation is therefore both a prediction task and an intermediate reasoning artifact.

The three-class design is also useful for downstream reasoning. A binary mask can indicate where a polyp is located, but it does not distinguish neoplastic from non-neoplastic regions. By preserving class information, the mask can provide richer evidence to the Polyp VQA Agent. This is particularly relevant when the user’s question asks about suspicious areas or class-related interpretation.

## 5.5 DeepLabV3+ for Polyp Segmentation

DeepLabV3+ is used as the core segmentation architecture for the Polyp Segmentation Agent. The target output is a dense three-class mask in which each pixel is assigned to background, neoplastic polyp region, or non-neoplastic polyp region. This setting requires both accurate localization and stable class prediction inside the lesion area.

**Architecture objective.** DeepLabV3+ is selected because it combines high-level semantic context with boundary recovery. Colonoscopy frames often contain lesions with variable size, weak contrast, specular highlights, and unclear tissue boundaries. A useful architecture must therefore capture multi-scale context while still preserving enough spatial detail for pixel-level prediction.

**Main components.** The architecture contains four important parts:

- **Backbone encoder:** extracts hierarchical feature maps from the input colonoscopy image. The deeper layers provide semantic information, while lower-level features retain edge and texture details.
- **Atrous convolution:** expands the receptive field without excessively reducing feature-map resolution. This is useful when a polyp boundary requires both local texture cues and wider mucosal context.
- **ASPP:** Atrous Spatial Pyramid Pooling applies parallel atrous convolution branches at multiple dilation rates, together with image-level pooling, to capture objects and contextual patterns at different scales.
- **Decoder:** combines upsampled high-level features with low-level features to recover spa-

tial detail and produce sharper segmentation boundaries.

The segmentation agent follows the DeepLabV3+ encoder-decoder design, summarized in Figure 9. The encoder first extracts hierarchical features from the input image through a deep convolutional backbone with atrous convolution. Atrous convolution is important because it enlarges the effective receptive field without reducing the feature-map resolution too aggressively. In colonoscopy frames, this is useful because the model must capture both local texture changes around the lesion boundary and broader contextual information from the surrounding mucosa.

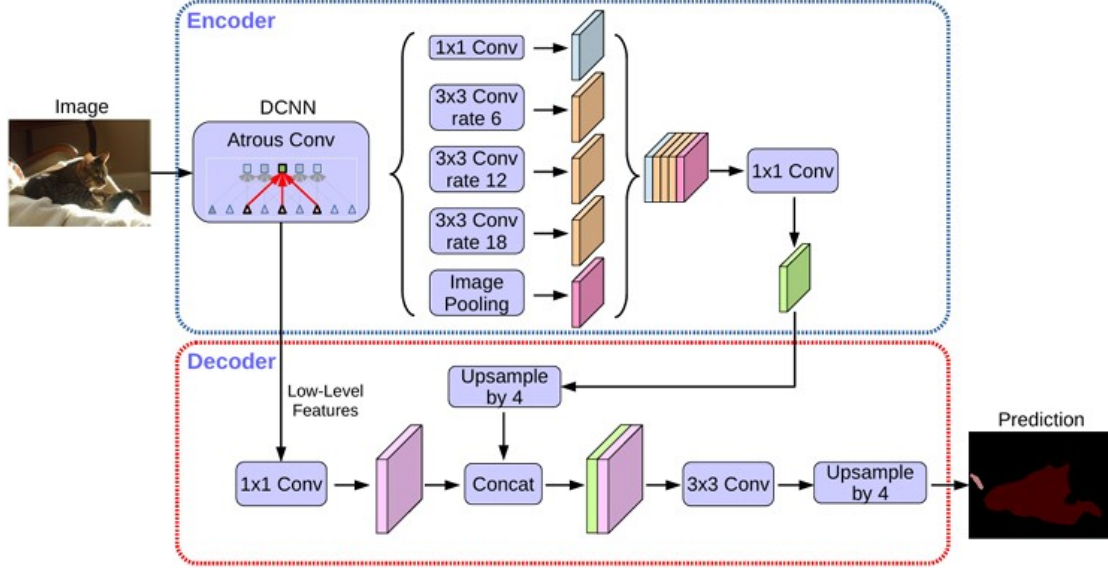


Figure 9: DeepLabV3+ encoder-decoder architecture with Atrous Spatial Pyramid Pooling

**Processing workflow.** Let  $X$  be the input colonoscopy image. The encoder first produces high-level semantic features:

$$F_e = ASPP(DCNN(X)). \quad (1)$$

The ASPP output is then upsampled and concatenated with low-level features  $F_l$  extracted from earlier layers:

$$F_d = \text{Concat}(\text{Upsample}(F_e), \text{Conv}_{1 \times 1}(F_l)). \quad (2)$$

The decoder refines the combined representation and produces the final pixel-wise prediction:

$$P = \text{Upsample}(\text{Conv}_{3 \times 3}(F_d)). \quad (3)$$

The dense prediction  $P$  is converted into the color mask used by the rest of the system. Blue denotes background, red denotes neoplastic regions, and green denotes non-neoplastic regions. This color convention is not only for visualization; it also provides the symbolic input used by the post-processing method and the segmentation-grounded Polyp VQA setting.

**Relevance to the thesis.** The encoder-ASPP-decoder design matches the needs of three-class polyp segmentation. ASPP helps detect lesions across scale variation, the decoder helps recover boundaries, and the class-wise mask preserves information needed for neoplasm characterization. However, a dense neural prediction can still contain small inconsistent class fragments inside a connected lesion region. The post-processing method below is therefore introduced as a region-level refinement step after DeepLabV3+ prediction.

## 5.6 Post-processing and Overlay Mask

Raw segmentation predictions may contain noisy regions or inconsistent labels inside a connected polyp region. To improve consistency, the system applies post-processing based on two main steps.

**Connected Component Analysis** identifies non-background connected regions in the mask. Each connected component is treated as a candidate object region. Let  $M(x, y)$  denote the raw predicted color label at pixel  $(x, y)$ , where blue represents background, red represents neoplastic tissue, and green represents non-neoplastic tissue. The first step converts the predicted mask into a binary region-of-interest map:

$$I(x, y) = \begin{cases} 1, & \text{if pixel } (x, y) \text{ belongs to a target region (non-blue),} \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Two pixels  $p = (x_1, y_1)$  and  $q = (x_2, y_2)$  are considered connected under an 8-neighborhood rule if:

$$|x_1 - x_2| \leq 1, \quad |y_1 - y_2| \leq 1, \quad I(p) = I(q) = 1. \quad (5)$$

The connected foreground pixels form a set of connected components:

$$\{R_i\}_{i=1}^N \quad \text{such that} \quad \forall (p, q) \in R_i, \quad p \leftrightarrow q, \quad (6)$$

where  $R_i$  is the  $i$ -th connected region,  $N$  is the number of detected connected regions, and  $p \leftrightarrow q$  means that there exists a path of connected foreground pixels between  $p$  and  $q$ . The region label map is then defined as:

$$L(x, y) = i \quad \text{if} \quad (x, y) \in R_i. \quad (7)$$

**Region-wise Color Dominance Refinement** checks the number of red and green pixels inside each connected component. For each connected region  $R_i$ , the number of pixels assigned to class  $k$  is computed as:

$$C_k(R_i) = \sum_{(x,y) \in R_i} \mathbf{1}[M(x, y) = k], \quad (8)$$

where  $k \in \{\text{red}, \text{green}\}$  and  $\mathbf{1}[\cdot]$  is an indicator function that returns 1 when the condition is true and 0 otherwise. The dominant class of the region is:

$$k^* = \arg \max_k C_k(R_i). \quad (9)$$

The same counts also provide a simple measure of how reliable the region-level decision is. Let  $|R_i|$  be the number of pixels in region  $R_i$ . The dominance ratio is:

$$\rho_i = \frac{\max_k C_k(R_i)}{|R_i|}. \quad (10)$$

A high  $\rho_i$  means that most pixels in the connected region already agree on the same class, so refinement mainly removes small noisy fragments. A low  $\rho_i$  indicates that the region is ambiguous because red and green occupy comparable portions. In that case, assigning a single class is more risky and should be interpreted carefully in qualitative analysis.

The refined mask  $M'$  is obtained by assigning the dominant class to all pixels in the connected component:

$$M'(x, y) = k^*, \quad \forall (x, y) \in R_i. \quad (11)$$

These equations make the assumption of the method explicit. Connected component analysis defines the spatial unit of refinement through  $R_i$ , while color dominance refinement defines the semantic decision through  $C_k(R_i)$  and  $k^*$ . The method therefore regularizes the existing prediction rather than redrawing the lesion from scratch. The formulation provides three practical insights:

- **Spatial insight from  $R_i$ :** refinement is performed per connected foreground object, not across the whole image. Therefore, separate candidate lesions are processed independently.
- **Semantic insight from  $C_k(R_i)$ :** the corrected class is determined by the class distribution already present in the raw prediction. The method regularizes the model output but does not introduce external clinical knowledge.
- **Reliability insight from  $\rho_i$ :** high-dominance regions are good candidates for refinement, while low-dominance regions should be treated as uncertain cases where the raw prediction itself is class-ambiguous.

Representative post-processing behavior is provided in Figure 10. Each row contains the original colonoscopy image, the ground-truth mask, the raw model prediction, and the processed prediction.

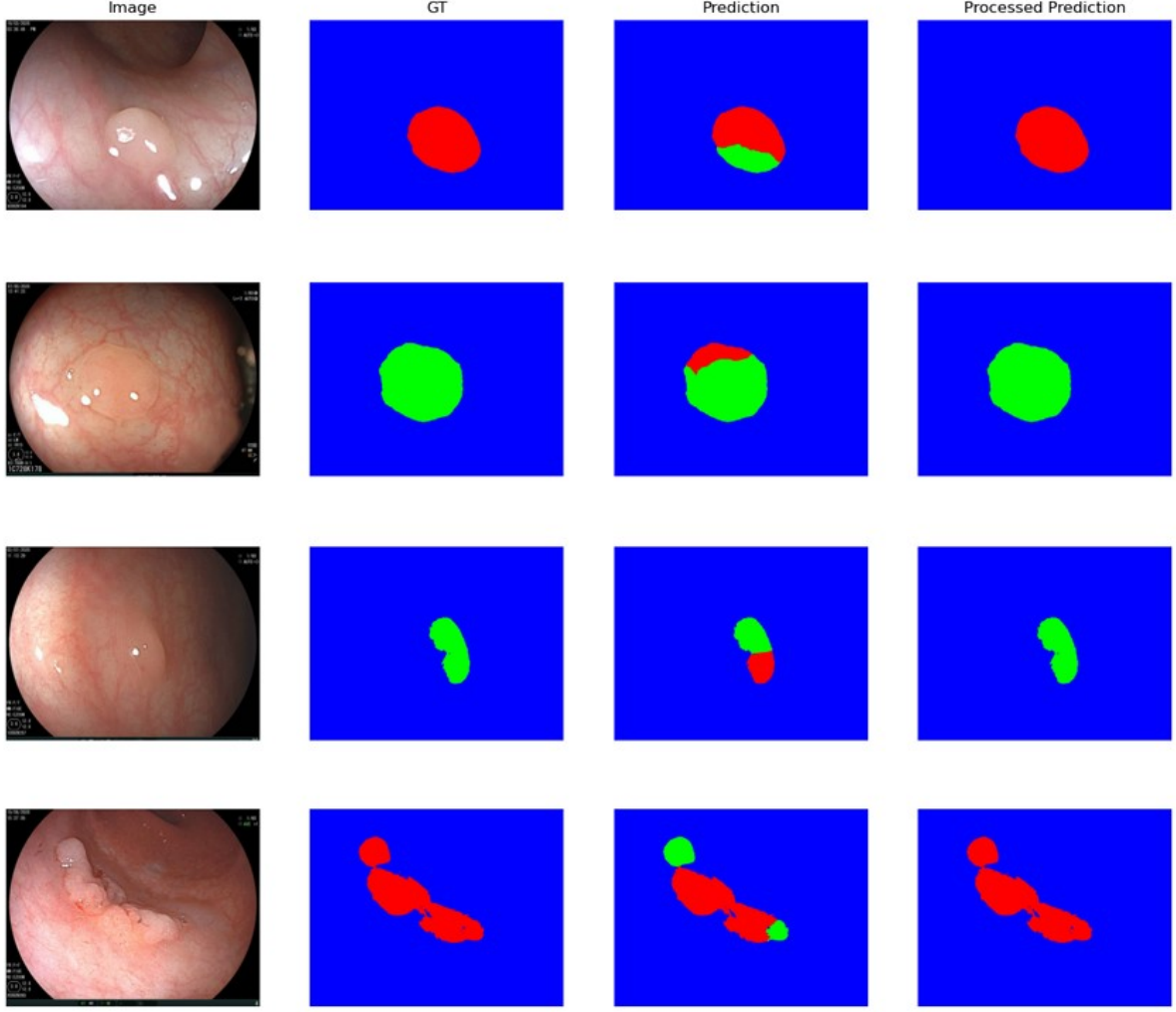


Figure 5: Comparison Result of The Masking Segmentation with The Technique

Figure 10: Visualization of raw and post-processed polyp segmentation masks

The examples show that post-processing is most effective when the segmentation model has already localized the lesion reasonably well but produces unstable class labels within the region. The technique is therefore a region-consistency correction step rather than a detector. It can improve mask readability and Dice Score, but it remains dependent on the quality of the raw prediction.

After refinement, the mask is overlaid on the original image. The overlay supports human inspection and provides a grounded visual input for the Polyp VQA Agent. This step is useful because a fragmented or mixed-color overlay may guide the VQA model toward inconsistent evidence, whereas a cleaner overlay is easier for both humans and models to interpret. However, the method is intentionally conservative: it improves regional consistency but does not create new lesion regions. If the initial model misses the polyp or detects a false foreground region, connected component analysis cannot correct that semantic error.

## 5.7 Polyp VQA Agent Design with LoRA Fine-tuned VLMs

The Polyp VQA Agent answers natural-language questions about colonoscopy polyp images. It is invoked when the Decision Agent detects that the user is asking a question about an endoscopic polyp image rather than requesting only segmentation or general image description. The agent receives an image-question input, formats it for the selected VLM endpoint, and returns a concise answer that should be medically relevant and visually grounded in the supplied

image.

The agent uses LoRA fine-tuned VLMs hosted through a configurable base URL. In this thesis, the VQA fine-tuning is designed for two 4B-scale backbones: MedGemma-4B-it and Qwen3.5-4B. This design allows the system to compare a medically oriented VLM backbone with a strong general instruction-following backbone while keeping the agent interface compatible with the rest of the workflow.

The dataset construction and experimental split for Polyp VQA are presented in Chapter 7. In this chapter, the focus is the agent role: converting image-question inputs into clinically relevant answers and integrating the fine-tuned VLM with the segmentation workflow.

The VQA agent is connected to the segmentation workflow through visual grounding. When a segmentation mask is available, the agent can receive both the original image and the overlay. This design gives the VLM explicit visual evidence about the region of interest while preserving the original image as the primary visual context. If no segmentation mask is available, the agent falls back to the original-image VQA setting.

This design gives the polyp workflow a two-stage reasoning structure: the Segmentation Agent localizes the region of interest, and the Polyp VQA Agent interprets the corresponding visual evidence in language. Segmentation without VQA is difficult for users to interpret, while VQA without segmentation may rely on less explicit visual evidence.

Using hosted fine-tuned VLMs through a base URL keeps the agent workflow independent from the physical deployment of the models. From the agent’s perspective, each Polyp VQA model is a callable service that receives an image-question input and returns an answer. The thesis focuses on the AI behavior of these models and their integration with segmentation, not on web backend details.

## 5.8 Segmentation-to-VQA Agent Cooperation

The cooperation between the Polyp Segmentation Agent and the Polyp VQA Agent illustrates the role of intermediate visual evidence in the multi-agent architecture. The Segmentation Agent answers a localization question: which pixels correspond to relevant polyp regions, and which predicted class is assigned to those regions? The VQA Agent answers a semantic question: how should the visual evidence be described in response to a natural-language question?

This separation produces the segmentation overlay as an intermediate evidence object. The overlay provides spatial grounding, improves interpretability, and connects pixel-level prediction with language-level reasoning. The VQA Agent does not need to infer the region of interest only from global image features; it receives an explicit visual cue generated by the segmentation pipeline.

| <b>Agent</b>                    | <b>Main contribution</b>                                                                            | <b>Limitation when used alone</b>                                                                              |
|---------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Polyp Segmentation Agent</b> | Produces pixel-level localization and class-colored masks for neoplastic or non-neoplastic regions. | A mask alone does not explain findings, answer user questions, or provide natural-language clinical reasoning. |
| <b>Polyp VQA Agent</b>          | Converts image-question input into a clinically relevant natural-language answer.                   | Without grounding, the VLM may rely on global context, dataset priors, or visually irrelevant regions.         |
| <b>Combined workflow</b>        | Uses the segmentation mask as visual evidence for VQA reasoning.                                    | The workflow depends on mask quality; an incorrect mask can mislead the VQA response.                          |

Table 8: Complementary roles of the Polyp Segmentation Agent and Polyp VQA Agent

Table 8 summarizes the complementarity between the two agents. The Segmentation Agent improves the precision of visual localization, while the VQA Agent improves the interpretability of the output. Their cooperation is especially relevant for questions such as whether a highlighted region is suspicious, where the polyp is located, or whether the visible region appears consistent with a neoplastic or non-neoplastic class label. These questions require both spatial evidence and language reasoning.

The cooperation also changes how the system should be evaluated. A segmentation model may achieve a high Dice Score but still produce overlays that are confusing for VQA, while a VQA model may produce fluent answers without using the highlighted region. Therefore, this thesis evaluates both the individual models and the segmentation-grounded VQA setting where the mask becomes part of the VQA input.

The Chapter 8 evaluation makes this design choice explicit in Table 15 and Table 16. On the mask-matched subset, Table 15 shows that adding visual grounding improves the LLM Judge score from 4.4269 to 4.5751 compared with the LoRA-only VQA baseline. The criterion-level analysis in Table 16 shows that the largest gains are in factual accuracy (+0.2033) and completeness (+0.1409). This pattern is consistent with the hypothesis that grounding helps the model select relevant visual information rather than merely changing the response style.

However, this cooperation is conditional rather than automatically beneficial. If the Segmentation Agent misses the lesion, highlights irrelevant mucosa, or assigns an incorrect class to the region, the VQA Agent may produce a more confident but wrong answer. This is a typical risk of pipeline-based reasoning: an upstream error can propagate downstream. For this reason, the segmentation overlay should be interpreted as an evidence cue, not as unquestionable ground truth. The VQA response should remain cautious, especially when the mask is small, fragmented, uncertain, or inconsistent with the visible image.

### 5.8.1 Example Interaction Pattern

An example workflow illustrates the interaction between the two agents. First, the user uploads a colonoscopy image and asks whether there is a visible polyp. The Decision Agent routes the request to the polyp workflow. The Segmentation Agent generates a mask and overlay, which identifies the suspected region. The Polyp VQA Agent then receives the original image, the overlay, and the user’s question. Instead of answering only from the raw image, it can refer to the highlighted region and provide a response such as: the image contains a region that should be reviewed, the highlighted area may correspond to a polyp-like structure, and a clinician should confirm the finding.

The user may then ask a follow-up question: “Why is this region suspicious?” In this case,

the system can reuse the same image-derived context. The follow-up question is textual, but it still depends on visual evidence. The workflow can route the question back to the Polyp VQA Agent or combine the VQA output with the Text Module for explanatory medical information. This illustrates cross-turn and cross-modal reasoning: the image is not treated as a one-time input, but as part of the ongoing consultation context.

This interaction pattern is useful for user-facing medical assistance because patients rarely ask only one complete question. They may upload an image, ask what it shows, ask whether the highlighted region is dangerous, ask what symptoms should be monitored, and ask when to see a doctor. The multi-agent workflow supports this behavior by preserving intermediate evidence and allowing later questions to reuse it.

## 5.9 Architectural Mechanism of Grounded Polyp VQA

**Input formulation and spatial prior.** The input of the segmentation-grounded VQA agent is not limited to a global endoscopic image. Instead, it is formulated as a composite signal that combines the original endoscopic image  $I_{\text{orig}}$ , a binary segmentation mask  $M_{\text{bin}} \in \{0, 1\}^{H \times W}$ , and the natural-language question  $q$ . The VQA input is therefore represented as:

$$X_{\text{vqa}} = (I_{\text{orig}}, M_{\text{bin}}, q). \quad (12)$$

In this formulation,  $M_{\text{bin}}$  functions as an ROI prior. Pixels with  $M_{\text{bin}}(x, y) = 1$  define the spatial support of the suspected lesion region, while pixels with  $M_{\text{bin}}(x, y) = 0$  are treated as background with respect to the grounding signal. The mask therefore provides a hard-attention prior: it explicitly constrains the spatial search space before clinical reasoning begins. Rather than allowing the VQA model to infer the relevant image region only from global visual context, the system supplies a geometric prior that indicates where visual evidence should be inspected.

This spatial prior is especially relevant in colonoscopy VQA because clinically meaningful structures often occupy only a small portion of the frame. Polyp boundaries, mucosal texture, surface irregularity, color contrast, and protrusion patterns may be local and visually subtle. By introducing  $M_{\text{bin}}$ , the system establishes a concrete spatial coordinate reference for the VQA process, reducing the risk that the model answers from irrelevant background tissue or dataset-level visual priors.

**Task decoupling through pure spatial grounding.** The use of a binary mask also creates a semantic-agnostic spatial reference. In this design, localization and clinical reasoning are separated into two different responsibilities. The Segmentation Agent determines the geometric region that should be inspected, while the VQA Agent remains responsible for interpreting the visual appearance of that region in response to the question.

This separation is important because the mask should not be treated as a complete semantic answer. It tells the VQA model where to look, but it does not by itself determine whether the surface is regular, whether the boundary is clear, whether the morphology appears suspicious, or whether the visible texture supports a particular response. For open-ended questions about shape, surface pattern, color, boundary, or lesion appearance, the model must extract and analyze visual features directly from  $I_{\text{orig}}$  inside the region constrained by  $M_{\text{bin}}$ .

Formally, the spatial support induced by the binary mask is:

$$\Omega_{\text{ROI}} = \{(x, y) \mid M_{\text{bin}}(x, y) = 1\}. \quad (13)$$

The grounded visual evidence is therefore not the mask itself, but the set of visual features extracted from  $I_{\text{orig}}$  under the spatial constraint defined by  $\Omega_{\text{ROI}}$ . In this sense,  $M_{\text{bin}}$  establishes



the spatial grounding signal, while  $I_{\text{orig}}$  remains the source of appearance-level evidence. The VQA model is forced to combine geometric guidance with direct visual feature extraction, rather than replacing image interpretation with the segmentation output.

**Optimization architecture and late cross-modal alignment.** The optimization strategy follows the same separation of responsibility. During fine-tuning, the visual encoder is kept frozen and functions as a static feature extractor. Given the original image and the grounded visual representation, the frozen vision encoder produces a stable set of visual tokens:

$$F_{\text{vis}} = \text{ViT}_{\text{frozen}}(I_{\text{orig}}, M_{\text{bin}}). \quad (14)$$

Because the vision backbone is not updated, the visual representation space is not distorted by the relatively small endoscopic VQA dataset. This is useful in the thesis setting because the available grounded samples are limited and the pretrained visual encoder already provides a general-purpose representation of image patches. Task adaptation is therefore concentrated in the language model through LoRA layers.

The grounded reasoning process can be summarized as:

$$\hat{a} = \text{LLM}_{\theta + \Delta_{\text{LoRA}}}(q, F_{\text{vis}}, M_{\text{bin}}), \quad (15)$$

where  $\hat{a}$  is the generated answer,  $\theta$  denotes the frozen or mostly preserved pretrained LLM parameters, and  $\Delta_{\text{LoRA}}$  denotes the trainable low-rank adaptation parameters. The LoRA layers implement parameter-efficient task adaptation by modifying the attention and projection behavior of the LLM while preserving most pretrained parameters.

Cross-modal fusion is therefore performed late, inside the LLM attention layers, rather than by retraining the visual encoder. Textual tokens from the question, visual tokens from  $F_{\text{vis}}$ , and spatial cues induced by  $M_{\text{bin}}$  are aligned during answer generation. This late cross-modal alignment allows the model to integrate the mask as a guidance signal without disrupting the original visual representation learned by the frozen encoder.

Under this formulation, the segmentation mask guides where the model should look, the original image provides the visual evidence that must be interpreted, and the question determines which visual properties are relevant. The grounded VQA setting should therefore be understood as spatially guided multimodal reasoning, not as replacing clinical visual interpretation with a segmentation label.

## 5.10 Original-image and Segmentation-grounded VQA Settings

The Polyp VQA model is studied under two input settings:

**Original image + question.** The model receives only the original colonoscopy image and the user’s question. This represents a VQA setting where the model must identify relevant visual evidence by itself.

**Original image + segmentation mask + question.** The model receives both the original image and the segmentation overlay. The mask acts as a visual grounding signal that guides the model toward the polyp region. This setting represents cooperation between the Polyp Segmentation Agent and the Polyp VQA Agent.

Visual grounding can improve answer quality when the mask correctly highlights clinically relevant regions, but it can also introduce risk if the segmentation mask is inaccurate. The comparison between these settings is therefore one of the central Vision Module experiments. The original-image setting measures the VLM’s baseline ability to extract relevant evidence

from the endoscopic frame, while the segmentation-grounded setting isolates the effect of adding explicit spatial evidence generated by the Segmentation Agent.

## 6 Chapter 6. Memory Layer: Long-Term Context for Medical Agents

### 6.1 Role of the Memory Layer

The Memory Layer provides persistent patient context for the multi-agent system. Its purpose is not to perform diagnosis, but to preserve clinically useful facts that may remain relevant across sessions, such as reported symptoms, allergies, medication use, chronic diseases, treatment reactions, warning signs, lifestyle factors, and communication preferences. This design addresses a limitation of ordinary chat history: short-term conversation state is useful inside one session, but it does not provide reliable continuity when the same patient returns with a related question later.

The proposed system uses Mem0 as the external memory management layer. Mem0 is designed around the idea of extracting salient facts from interactions, storing them with embeddings and metadata, and later retrieving relevant memories for a new query [4, 5]. In this thesis, Mem0 is treated as a memory substrate rather than as a medical reasoning model. It does not replace LangGraph, RAG, Knowledge Graph reasoning, image analysis, or patient database storage. Instead, it provides an additional patient-context channel that can improve routing, personalization, and continuity.

A key design principle is separation of responsibility. LangGraph manages workflow state and agent coordination; RAG and the Knowledge Graph provide external medical knowledge; the Memory Layer stores reusable patient-specific facts; and task-specific agents generate outputs under safety constraints. This separation prevents long-term memory from becoming an uncontrolled medical knowledge base or a hidden diagnostic record. In particular, the system stores memory as reported or source-attributed facts rather than as confirmed clinical truth.

### 6.2 Integration Architecture

The implemented architecture wraps Mem0 inside a patient-scoped service rather than calling the library directly from every agent. The service is implemented in `agents/patient_memory_service/memory_service.py`. It lazily constructs a Mem0 Memory instance, configures Qdrant as the vector store, attaches a LangChain LLM for fact extraction, attaches a LangChain embedding model, and passes custom medical memory instructions to Mem0. The service also standardizes metadata for patient scope, session scope, condition type, source, status, and timestamps.

The memory service exposes four main operations:

- **Explicit condition write:** the condition-write endpoint stores a structured clinical fact with no extra inference. This is used when the system already has a fact that should be preserved exactly.
- **Conversation-based write:** `add_conversation` stores chat turns with `infer=True`, allowing Mem0 to extract durable facts from the interaction.
- **Semantic search:** `search` retrieves top- $k$  memories using a natural-language query, patient filters, optional condition filters, and a relevance threshold.
- **Governance operations:** list and delete endpoints support memory review, correction workflows, and patient-scoped deletion.

The service is also exposed through the FastAPI router `/api/patient-memory`. This makes

the Memory Layer usable both inside the LangGraph workflow and as a separate service for debugging, inspection, or future administrative tools. In the current application, `app.py` includes this router, while the graph itself calls the Python service directly.

At the orchestration level, the integration follows a retrieval-before-reasoning and write-after-guardrails pattern. First, the LangGraph node `retrieve_patient_memory` searches for relevant patient memory before the Decision Agent selects an agent. The formatted memory context is then inserted into the routing prompt and into downstream agent prompts as supporting context. After answer generation, human-validation handling, and output guardrails, the `write_patient_memory` node considers whether the current interaction contains durable facts worth storing. The graph ordering is therefore:

```
analyze_input → retrieve_patient_memory → route_to_agent → task_agent →
check_validation → apply_guardrails → write_patient_memory.
```

This placement is intentional. Retrieval before routing allows memory to influence agent selection, while writing after guardrails reduces the risk of storing irrelevant, unsafe, or rejected content.

### 6.3 Technical Pipeline

The technical pipeline can be formalized as a patient-scoped memory function. Let  $p$  denote a patient identifier,  $r$  a session or run identifier,  $q_t$  the current user query at turn  $t$ ,  $H_t$  the short-term conversation history, and  $M_p$  the set of stored memories for patient  $p$ . Each memory item is represented as:

$$m_i = (x_i, e_i, a_i),$$

where  $x_i$  is the natural-language memory text,  $e_i = \text{Embed}(x_i)$  is its vector representation, and  $a_i$  is a metadata dictionary containing source, type, status, timestamps, and provenance fields. For a new query  $q_t$ , memory retrieval computes:

$$\text{Retrieve}(q_t, p) = \text{TopK}_{m_i \in M_p} (\text{sim}(\text{Embed}(q_t), e_i)),$$

subject to metadata filters such as `user_id=p`, `agent_id`, `run_id`, `condition_type`, and `status`. This corresponds to the implemented call `memory.search(query, filters, top_k, threshold)`.

**Memory construction.** The system supports two write modes. In explicit condition writing, the application constructs a memory string from a validated structured payload and stores it without LLM inference. In conversation writing, the system submits user and assistant turns to Mem0 and lets the configured LLM extract concise durable facts according to the medical memory policy. In both cases, Mem0 embeds the resulting memory and stores it in Qdrant with metadata and history tracking.

Listing 1: Patient-scoped memory retrieval before agent routing

```
1 | Input: patient_id p, query q, memory_enabled flag, top_k K, threshold
   | tau
2 | Output: formatted memory context C and raw memory items R
3 |
4 | if memory_enabled is false or q is empty then
5 |     return C = "", R = []
6 | end if
```

```

7 |
8 | filters <- {
9 |     user_id: p,
10 |    agent_id: PATIENT_MEMORY_AGENT_ID
11 | }
12 |
13 | R <- Mem0.search(
14 |     query = q,
15 |     filters = filters,
16 |     top_k = K,
17 |     threshold = tau
18 | )
19 |
20 | C <- []
21 | for each memory item m in first five items of R do
22 |     source <- m.metadata.source or m.metadata.memory_kind
23 |     if source in {ai_image_analysis, assistant, ai} then
24 |         prefix <- "AI-recorded but unconfirmed"
25 |     else
26 |         prefix <- "Patient-reported"
27 |     end if
28 |     append format(prefix, m.text, source, m.score) to C
29 | end for
30 |
31 | return C, R

```

Algorithm 1 corresponds to the retrieval and formatting helpers in the implementation. A clinically important detail is that retrieved memory is source-labeled before being added to the prompt. AI-derived image analysis is explicitly marked as unconfirmed, while patient-reported information is marked as reported by the patient. This prevents the model from silently converting memory into confirmed diagnosis.

Listing 2: Post-response memory writing with medical safety constraints

```

1 | Input: graph state S containing patient_id, session_id, user input,
   | agent output
2 | Output: updated state S
3 |
4 | if S.memory_enabled is false then
5 |     return S
6 | end if
7 |
8 | input_text <- text(S.current_input)
9 | output_text <- text(S.output)
10 |
11 | if input_text and output_text are empty then
12 |     return S
13 | end if
14 |
15 | if S.agent_name == NON_MEDICAL_FILTER then
16 |     return S
17 | end if
18 |
19 | if S.current_input contains a medical image and output_text is not
   | empty then

```

```

20     fact <- "AI-recorded medical image analysis, requires clinician
      confirmation: "
21       + shorten(output_text)
22     Mem0.add(
23       messages = [{role: user, content: fact}],
24       user_id = S.patient_id,
25       agent_id = PATIENT_MEMORY_AGENT_ID,
26       run_id = S.session_id,
27       metadata = {source: ai_image_analysis, validated: false},
28       infer = false
29     )
30     return S
31   end if
32
33   messages_to_store <- []
34   if input_text is not empty then
35     append {role: user, content: shorten(input_text)} to
      messages_to_store
36   end if
37   if output_text is not empty and input is not a validation result then
38     append {role: assistant, content: shorten(output_text)} to
      messages_to_store
39   end if
40
41   if messages_to_store is not empty then
42     Mem0.add(
43       messages = messages_to_store,
44       user_id = S.patient_id,
45       agent_id = PATIENT_MEMORY_AGENT_ID,
46       run_id = S.session_id,
47       metadata = {source: chat, stored_by:
      langgraph_write_patient_memory},
48       infer = true
49     )
50   end if
51
52   return S

```

Algorithm 2 corresponds to the `write_patient_memory` node. It distinguishes image-derived findings from ordinary chat-derived facts. Image findings are stored explicitly with `infer=False` and `validated=False`; chat turns are passed to Mem0 with `infer=True` so that the configured extraction LLM can extract concise durable memories. The memory policy in `config.py` instructs the system to store only durable patient-condition facts, preserve Vietnamese when appropriate, avoid identifiers and secrets, avoid speculative diagnoses, and mark uncertain information as reported or uncertain.

## 6.4 Impact on Agent Reasoning

Long-term memory affects the system at the level of control flow and context construction rather than by directly deciding the final medical answer.

**Decision Agent.** Memory can improve routing. The decision prompt contains the current query, recent conversation context, image status, image type, and retrieved long-term patient memory. For example, if the user asks “Can I take this medicine?” and retrieved memory

indicates a penicillin allergy, the query should be routed toward medical reasoning rather than casual conversation.

**Conversation Agent.** Memory can reduce repeated questions. If prior memory already contains chronic disease history, medication use, allergy, or lifestyle factors, the agent can ask more targeted follow-up questions instead of repeatedly collecting the same background information.

**KG/RAG Agents.** Memory can personalize retrieval. Patient context can guide query expansion and evidence synthesis, for example by making medication interactions, risk factors, or chronic disease context more salient. However, RAG and Knowledge Graph evidence remain responsible for medical grounding.

**Vision-related agents.** Memory can provide background context, but image evidence must remain primary. If memory indicates a prior history of colon polyps, this may support routing to the polyp workflow. Nevertheless, segmentation masks, VQA outputs, and the current image should remain the main basis for image-specific interpretation.

**Human validation and guardrails.** Memory is inserted before reasoning, but memory writing is performed after output guardrails. This ordering helps avoid storing content from rejected non-medical image requests and keeps memory updates closer to the final system behavior.

## 6.5 Implementation Correspondence

Table 9 maps the conceptual Memory Layer to the current codebase.

| Conceptual component | Implementation                                        | Function in the system                                                                                             |
|----------------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Memory configuration | <code>patient_memory_service/config.py</code>         | Defines Qdrant paths, embedding dimensions, agent identifier, LLM parameters, and medical extraction instructions. |
| Mem0 wrapper         | <code>patient_memory_service/memory_service.py</code> | Builds Mem0 with Qdrant, LangChain LLM, LangChain embedder, metadata, and result normalization.                    |
| Memory API           | <code>patient_memory_service/api.py</code>            | Provides health, condition write, conversation write, search, list, and delete endpoints.                          |
| Retrieval node       | <code>retrieve_patient_memory</code>                  | Retrieves patient memory before routing and attaches formatted context to graph state.                             |
| Routing integration  | <code>route_to_agent</code>                           | Adds memory context to the Decision Agent input.                                                                   |
| Prompt boundary      | memory context helper and agent prompts               | Inserts memory as supporting context only, not as confirmed diagnosis.                                             |
| Write node           | <code>write_patient_memory</code>                     | Stores durable facts after validation and guardrails.                                                              |
| Evaluation script    | <code>benchmark/test_memory_layer.py</code>           | Measures extraction, retrieval, routing effect, repeated-question avoidance, answer quality, and latency.          |

Table 9: Mapping between the Memory Layer design and the current implementation

## 6.6 Safety Constraints and Limitations

Memory is not a diagnosis. Patient-reported facts should be represented as reported information, such as “the user reports...” or “the patient states...”. AI-inferred conclusions should be marked as uncertain unless validated. This distinction is particularly important because retrieval can make an old memory appear authoritative even when the original fact was incomplete, outdated, or only self-reported.

The memory layer should not store raw images, base64 image content, API keys, phone numbers, addresses, or unnecessary personally identifiable information. The implemented custom instructions explicitly reject casual chat, unrelated preferences, secrets, raw identifiers, raw image data, speculative diagnoses, assistant disclaimers, and generic medical advice.

Memory can also become outdated or incorrect, so it must be used as supporting context rather than authoritative truth. A medication that was active in one session may be discontinued later; a symptom may resolve; a suspected disease may be ruled out. Future deployment would require stronger governance mechanisms, including explicit user review, correction, deletion, source tracking, recency weighting, confidence marking, and clinician validation workflows. These mechanisms are especially important when memory is used across multiple sessions because continuity improves usefulness only when the system also supports correction and forgetting.

## 7 Chapter 7. Data and Experimental Setup

### 7.1 VimedAQA Dataset

VimedAQA is used as the Vietnamese medical question-answering source for the RAG component [8]. In the local index used by this thesis, the dataset is represented by 44,228 JSONL records across six groups: body part, disease, drug, and medicine-related passages. Each record contains a medical context, title, keyword, topic identifier, source URL, and author metadata. This structure makes the dataset suitable for retrieval-oriented medical QA rather than direct model fine-tuning.

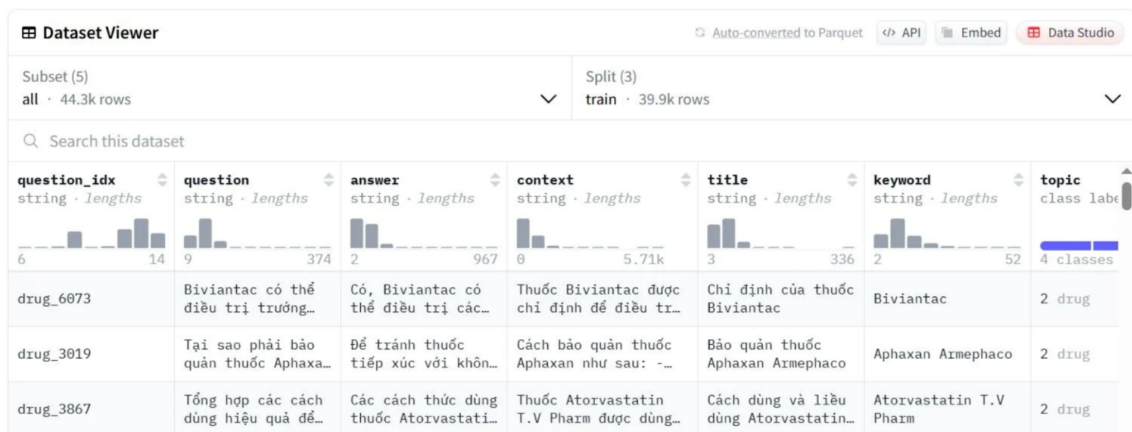


Figure 11: Medical QA dataset overview used for retrieval-oriented experiments

| VimedAQA Group | Number of Records |
|----------------|-------------------|
| Body part      | 4,956             |
| Disease 1      | 7,500             |
| Disease 2      | 8,154             |
| Drug           | 9,754             |
| Medicine 1     | 7,500             |
| Medicine 2     | 6,364             |
| Total          | 44,228            |

Table 10: Local VimedAQA record distribution used for RAG indexing

For retrieval-based reasoning, each record is converted into a retrievable passage. During inference, the user query is embedded and compared with stored passages, and the most relevant contexts are supplied to the answer generator. This setup allows the system to reuse Vietnamese medical information while keeping response generation separate from data indexing.

The quality of this component depends on preprocessing and retrieval design. Noisy passages, duplicated records, or overly broad chunks can reduce retrieval precision. Therefore, the dataset is treated as a knowledge source that requires cleaning, normalization, consistent chunking, and metadata-aware indexing.

## 7.2 VietMedKG

VietMedKG is used as the structured medical knowledge source for the Knowledge Graph Agent. In the local data used by this thesis, the graph source table contains 8,900 disease records with fields such as disease name, description, disease type, prevention, causes, symptoms, risk groups, comorbidities, treatment methods, clinical department, tests, nutrition advice, and medication-related information. These fields are converted into structured facts for graph-based retrieval and reasoning.

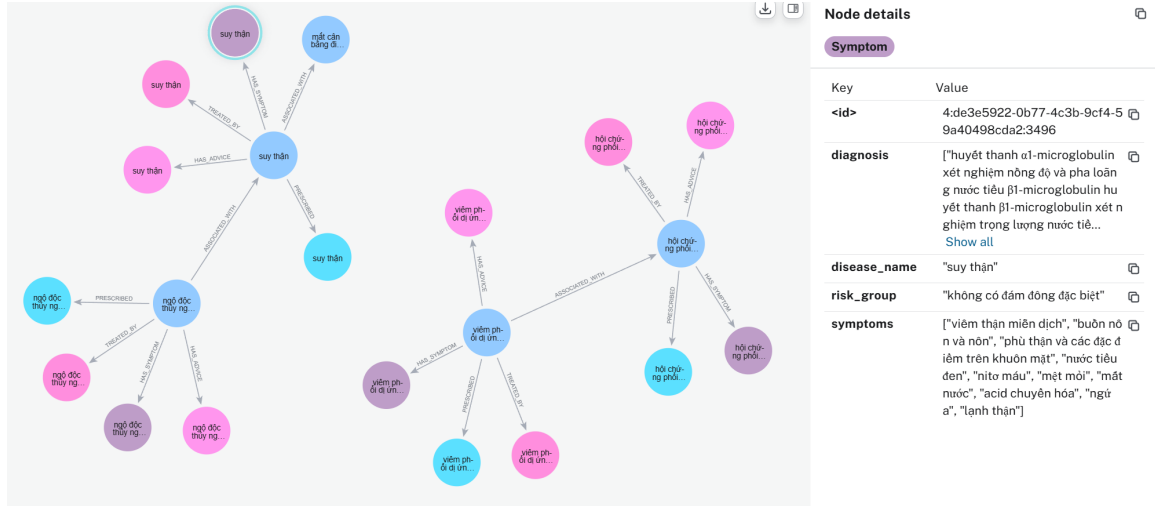


Figure 12: Overview of the VietMedKG data source used by the Knowledge Graph Agent

VietMedKG complements RAG by supporting relation-oriented reasoning. For example, the system can retrieve symptoms associated with a disease or identify diseases related to a symptom set. This makes the graph particularly useful for one-hop medical relation questions and for checking whether a generated answer is consistent with structured medical facts.

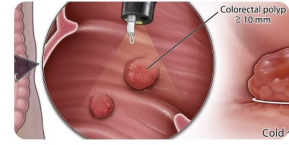


### 7.3 BKAI-IGH NeoPolyp-Small Dataset

BKAI-IGH NeoPolyp-Small is used for the polyp segmentation experiments. The dataset contains colonoscopy images with pixel-level annotations for neoplastic and non-neoplastic polyp regions. In this thesis, the segmentation task is formulated as a three-class problem: background, neoplastic polyp region, and non-neoplastic polyp region. This makes the dataset suitable for evaluating both lesion localization and class-aware mask prediction.

#### BKAI-IGH NeoPolyp

A dataset for colonoscopy polyp segmentation and neoplasm characterization



Overview **Data** Code Models Discussion Leaderboard Rules Team Submissions

#### Dataset Description

##### File description:

- **test:** test images
- **train:** training images
- **train\_gt:** ground-truths of training images
- **sample\_submission.csv:** sample submission

##### Ground-truth explanation

- Red color: neoplastic polyps
- Green color: non-neoplastic polyps
- Black color: background

##### Files

2201 files

##### Size

375.42 MB

##### Type

jpeg, csv

##### License

[Subject to Competition Rules](#)

Figure 13: Example images and annotations from the BKAI-IGH NeoPolyp-Small dataset

The annotations in Figure 13 serve as ground truth for Dice Score evaluation and for analyzing whether post-processing improves mask consistency. The three-class setting is important because a mask is not only a geometric localization result; it also carries semantic information that may be used by the VQA agent. Therefore, segmentation errors can affect both direct mask interpretation and downstream visual grounding.

### 7.4 Kvasir-VQA-X1 Dataset

Kvasir-VQA-X1 is used as the primary dataset for LoRA fine-tuning and evaluation of the Polyp VQA VLM. It is a gastrointestinal endoscopy visual question answering dataset organized as image-question-answer triples [38]. The public dataset extends Kvasir-VQA with 159,549 new question-answer pairs designed for medical reasoning and robustness evaluation. In the local training split used by this thesis, the full VQA setting contains 6,212 original endoscopic images and 143,594 image-question-answer samples.

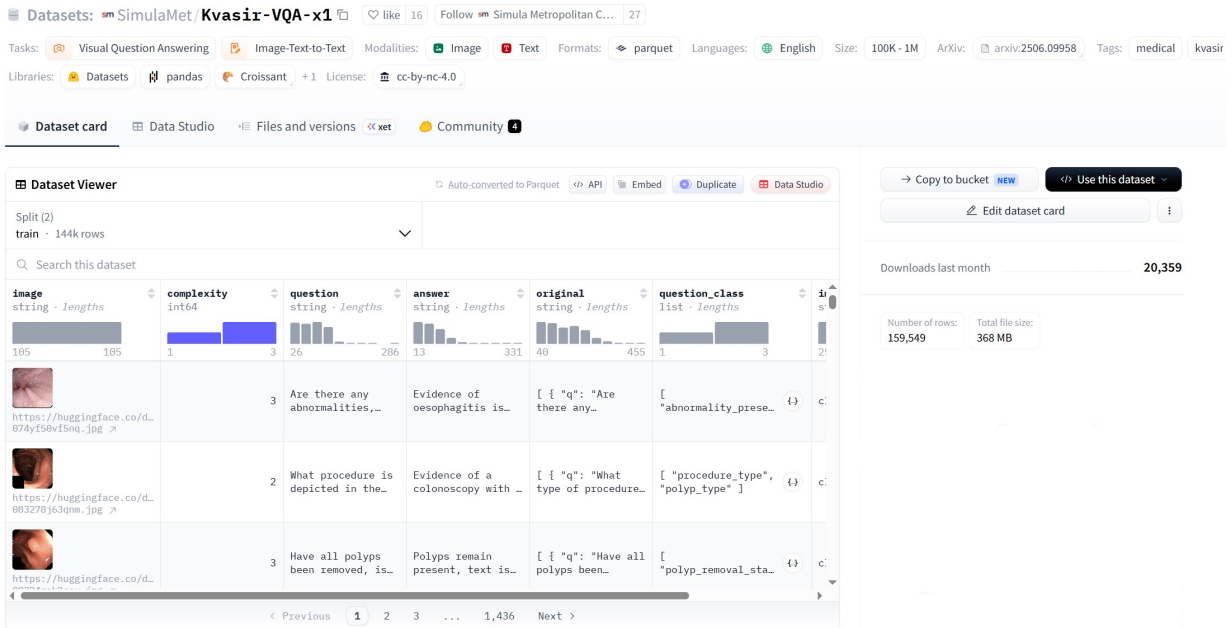


Figure 14: Example structure of the Kvasir-VQA-X1 dataset for gastrointestinal endoscopy VQA

This structure is suitable for the thesis because the model must connect endoscopic visual evidence with a natural-language question rather than only predict an image label or segmentation mask. Kvasir-VQA-X1 is used in two experimental formats:

- **Original-image VQA:** the model receives the endoscopic image and the question;
- **Visual-grounded VQA:** the model receives the original image, a segmentation mask or overlay, and the question.

The first format evaluates direct visual reasoning from the original frame. The second evaluates whether segmentation-derived grounding improves downstream VQA by giving the model an explicit cue about the region of interest.

The full Kvasir-VQA-X1 training split contains 6,212 original endoscopic images and 143,594 image-question-answer samples. However, segmentation-based visual grounding can only be applied when an image has a matched segmentation mask. After filtering for mask availability, 1,336 images remain with usable segmentation masks, corresponding to 42,382 training samples. Table 11 summarizes this filtering step.

| Training Data Setting                 | Number of Images | Number of Samples |
|---------------------------------------|------------------|-------------------|
| Original Kvasir-VQA-X1 training data  | 6,212            | 143,594           |
| Mask-filtered visual grounding subset | 1,336            | 42,382            |

Table 11: Kvasir-VQA-X1 training data before and after segmentation-mask filtering

This filtering step is necessary because visual grounding requires both the original image and a corresponding segmentation mask. Therefore, improvements from visual grounding should be compared against a LoRA-only baseline evaluated on the same mask-matched subset, not against a model evaluated on the full dataset. Evaluation data are kept separate from training data to avoid image-level or question-level leakage, and preprocessing is kept conservative to preserve clinically relevant visual evidence.

## 7.5 Polyp VQA Fine-tuning Setup

The Polyp VQA models are fine-tuned with LoRA. The training objective is to generate correct and clinically relevant answers from image-question inputs. The fine-tuning setup com-

pares two base-model families, MedGemma-4B-it and Qwen3.5-4B, under both original-image input and visual-grounded input.

Table 12 summarizes the main hyperparameters used for LoRA fine-tuning. The same setup is used for MedGemma-4B-it and Qwen3.5-4B in the original-image setting, with the corresponding base model changed according to the experiment.

| Hyperparameter                   | Original-image LoRA        | Visual-grounded LoRA              |
|----------------------------------|----------------------------|-----------------------------------|
| Training dataset                 | Kvasir-VQA-X1 train        | Mask-filtered Kvasir-VQA-X1 train |
| Validation dataset               | Kvasir-VQA-X1 test-1000    | Mask-matched validation subset    |
| Base models                      | MedGemma-4B-it, Qwen3.5-4B | MedGemma-4B-it                    |
| Maximum sequence length          | 512                        | 1024                              |
| Torch dtype                      | bfloat16                   | bfloat16                          |
| Number of epochs                 | 1                          | 3                                 |
| Train batch size per device      | 4                          | 4                                 |
| Evaluation batch size per device | 4                          | 4                                 |
| Gradient accumulation steps      | 4                          | 4                                 |
| Learning rate                    | $2 \times 10^{-5}$         | $2 \times 10^{-5}$                |
| Learning-rate scheduler          | linear                     | linear                            |
| Warmup ratio                     | 0.03                       | 0.03                              |
| Weight decay                     | 0.01                       | 0.01                              |
| LoRA rank                        | 16                         | 16                                |
| LoRA alpha                       | 64                         | 64                                |
| Freeze vision encoder            | true                       | true                              |
| Gradient checkpointing           | true                       | true                              |
| Best-model metric                | eval token accuracy        | eval token accuracy               |
| Checkpoint save interval         | 1000 steps                 | 1000 steps                        |
| Maximum saved checkpoints        | 2                          | 2                                 |
| Logging interval                 | 20 steps                   | 20 steps                          |
| Data loading workers             | 2                          | 2                                 |
| Dataset preprocessing processes  | 2                          | 2                                 |

Table 12: LoRA fine-tuning hyperparameters for Polyp VQA

The visual-grounded setting uses a longer maximum sequence length of 1024 because each example contains two visual inputs: the original endoscopic image and the segmentation-derived grounding image. In contrast, the original-image LoRA setting uses a maximum length of 512 because it only includes one image and the question-answer text.

The number of epochs is also different between the two settings. Original-image LoRA is trained for one epoch on the full training set, while the visual-grounded setting is trained for three epochs on the filtered subset. Since the mask-filtered subset is substantially smaller than the full VQA training set, the additional epochs are used to keep the number of training iterations more comparable between the two training strategies. This reduces optimization-step count as a potential confounding variable when comparing original-image fine-tuning and visual-grounded fine-tuning. Training and validation loss curves did not show signs of overfitting across the visual-grounded epochs, supporting the use of additional epochs on the smaller subset.

The original-image setting trains each model to answer from the endoscopic frame and question only. The visual-grounded setting extends the input with segmentation-derived evidence, such as a mask or overlay. To make the comparison fair, each base model should be evaluated under both input settings with the same LoRA strategy and the same evaluation split. This separates two effects: the difference between base-model backbones and the effect of explicit segmentation guidance.

The output format should be consistent across training and evaluation. Short factual answers are easier to evaluate with lexical metrics, while longer explanatory answers may better reflect

clinical reasoning but are harder to score with BLEU or ROUGE. For this reason, LLM Judge is included as a complementary evaluation method.

## 7.6 Memory Evaluation Scenarios

The memory layer should be evaluated using multi-session Vietnamese medical scenarios. Representative scenarios include:

- a user reporting a medication allergy in one session and asking about a drug later;
- a user with a chronic condition asking follow-up symptom questions;
- repeated symptoms across sessions;
- medication use that should influence later advice;
- follow-up questions after medical image analysis.

## 7.7 Evaluation Metrics

Different components require different metrics.

**Text Module.** One-Hop Accuracy and Multi-Hop Awareness Score measure direct reasoning and awareness of missing information.

**Segmentation.** Dice Score measures overlap between predicted masks and ground-truth masks.

**Polyp VQA.** BLEU, ROUGE-L, and LLM Judge evaluate answer quality. BLEU and ROUGE-L measure textual overlap, while LLM Judge evaluates semantic correctness, completeness, visual grounding, and safety.

**Memory Layer.** Memory retrieval relevance, response consistency, repeated-question reduction, routing accuracy, and latency can be used to evaluate the impact of long-term memory.

## 7.8 Experimental Protocol

The experimental protocol follows a component-wise and system-wise evaluation design. Component-wise evaluation measures each technical module with metrics suitable for its output format. The Text Module is evaluated with one-hop and multi-hop reasoning benchmarks. The Segmentation Agent is evaluated with Dice Score. The Polyp VQA Agent is evaluated on Kvasir-VQA-X1 using lexical and judge-based metrics. The Memory Layer is evaluated through multi-session scenarios.

System-wise evaluation focuses on interaction behavior. This includes whether the Decision Agent selects the correct processing path, whether the Text Module asks for missing information, whether visual grounding improves or harms VQA, and whether memory improves continuity without injecting irrelevant context. This distinction is important because a multi-agent assistant can fail even when individual components perform reasonably well. For example, a good segmentation model does not help if the image query is routed to the wrong agent.

For all experiments, qualitative analysis should accompany numerical metrics. Medical AI outputs are difficult to evaluate with a single number because correctness, safety, clarity, and uncertainty handling are all important. The report should therefore include representative successes and failures, especially for high-risk symptom questions, incorrect retrieval cases, segmentation failures, and VQA examples where the mask changes the answer.

## 8 Chapter 8. Results and Analysis

### 8.1 Text Module Results and Analysis

The Text Module is evaluated using one-hop and multi-hop medical reasoning benchmarks. Table 13 summarizes the comparison.

| Model Variant      | One-Hop Accuracy | Multi-Hop Awareness Score |
|--------------------|------------------|---------------------------|
| <b>KG + MedLLM</b> | <b>86.95%</b>    | <b>70.83%</b>             |
| RAG + MedLLM       | 67.97%           | 40.38%                    |
| MedLLM             | 60.87%           | 50.72%                    |

Table 13: Text Module benchmark results

The KG + MedLLM setting achieves the best performance in both One-Hop Accuracy and Multi-Hop Awareness Score. This suggests that structured medical relations help the model reason more accurately when the task depends on disease-symptom links or explicit medical relationships.

RAG + MedLLM performs better than the base model on one-hop reasoning but is weaker on Multi-Hop Awareness. This indicates that retrieved text is useful for explanation, but it may not always help the model recognize when information is incomplete. The base MedLLM setting lacks external grounding and therefore performs worse on structured reasoning.

The difference between One-Hop Accuracy and Multi-Hop Awareness is important. One-Hop Accuracy measures whether the system can answer when the required relation is available. Multi-Hop Awareness measures whether the system understands that a direct answer may be unsafe when key information is missing. A medical assistant should perform well on both dimensions. High answer accuracy alone is not sufficient if the model answers prematurely in incomplete clinical scenarios.

The stronger performance of KG + MedLLM suggests that explicit relations help the model identify clinically relevant links more reliably than text retrieval alone. However, the result should not be interpreted as showing that KG is always superior to RAG. KG is stronger for structured relation reasoning, while RAG remains useful for explanatory responses and broader medical education. The practical conclusion is that the two sources should be coordinated rather than treated as mutually exclusive.

The lower Multi-Hop Awareness Score of RAG + MedLLM suggests that adding retrieved documents may encourage answer generation even when the user query lacks enough clinical detail. This is a common risk in retrieval-augmented systems: retrieved evidence can make the model appear grounded while the original question remains underspecified. The MedGemma CoT information-sufficiency step is therefore necessary after KG/RAG context selection and before final response synthesis.

The base MedLLM result in Table 13 shows that a language model alone can answer some medical questions but performs worse when structured external knowledge is required. Therefore, the benchmark motivates placing the LLM inside a retrieval and graph-augmented pipeline rather than using it as the only reasoning source.

### 8.2 Polyp Segmentation Results and Analysis

Table 14 presents Dice Score results for several segmentation models with and without post-processing.

| Models                  | DICE Score         |                 |
|-------------------------|--------------------|-----------------|
|                         | Without Processing | With Processing |
| PraNet                  | 80.02%             | 82.94%          |
| DeepLabV3+ (RegNetx320) | <b>82.56%</b>      | <b>86.33%</b>   |
| DeepLabV3+ (ResNet50)   | 81.12%             | 85.62%          |
| EMCAD (PVT2-b2)         | 80.93%             | 83.76%          |
| U-Net                   | 75.95%             | -               |
| U-Net++                 | 77.54%             | -               |

Table 14: Dice Score comparison for polyp segmentation models

Post-processing improves segmentation consistency across the evaluated models. The RegNetx320 variant achieves the strongest result, increasing from 82.56% to 86.33% Dice Score. The ResNet50 variant also improves from 81.12% to 85.62%. These gains indicate that region-level refinement is useful even when the base segmentation model is already strong.

The improvement is consistent with the method described in Chapter 5. Connected component analysis changes the interpretation of the mask from isolated pixel predictions to region-level candidates, and region-wise color dominance refinement reduces mixed red/green class fragments inside a connected lesion. This is especially relevant in the BKAI-IGH NeoPolyp setting because a pixel can be spatially inside the lesion but still counted as wrong if its neoplastic/non-neoplastic label is incorrect.

The qualitative examples in Figure 10 align with the quantitative trend in Table 14. The largest gains appear when the raw prediction is close to the correct region but contains minority-class artifacts. In these cases,  $C_k(R_i)$  identifies the dominant class, and the update rule for  $M'(x, y)$  improves overlap by regularizing the label inside the connected component.

The method should therefore be interpreted as a consistency refinement module, not as a replacement for the segmentation model. It cannot recover a missed lesion, and it may incorrectly merge touching regions that actually belong to different classes. For downstream Polyp VQA, its main benefit is a clearer overlay for visual grounding and human review; its main risk is that an incorrect refined mask may guide the VQA model toward wrong evidence. This motivates the separate evaluation of original-image VQA and segmentation-grounded VQA.

### 8.3 Polyp VQA Fine-tuning Results and Analysis on Kvasir-VQA-X1

The Polyp VQA VLM is evaluated using BLEU-4, ROUGE-L, and an LLM-as-a-judge protocol. BLEU-4 and ROUGE-L measure lexical overlap with the reference answer, while the judge-based score is used to assess semantic and clinical answer quality. The judge model is `google/medgemma-27b-it`. For each prediction, the judge receives the question, ground-truth answer, and model prediction, and returns four scores from 1 to 5: factual accuracy, completeness, clinical relevance, and no hallucination. The final weighted score is computed as:

$$\begin{aligned} \text{Score} = & 0.40 \times \text{Factual Accuracy} + 0.40 \times \text{Completeness} \\ & + 0.10 \times \text{Clinical Relevance} + 0.10 \times \text{No Hallucination}. \end{aligned}$$

This weighting gives the highest importance to factual correctness and coverage of the reference answer, while still accounting for clinical language quality and hallucination control. This is appropriate for medical VQA because an answer that is fluent but factually wrong or incomplete should not be considered reliable.

| Model Setting                                                 | Samples | BLEU-4 | ROUGE-L | LLM Judge |
|---------------------------------------------------------------|---------|--------|---------|-----------|
| MedGemma-4B-it base                                           | 1000    | 0.0055 | 0.1154  | 2.7521    |
| MedGemma-4B-it + LoRA fine-tuning                             | 1000    | 0.3657 | 0.6936  | 4.6351    |
| Qwen3.5-4B base                                               | 1000    | 0.0261 | 0.2078  | 3.4720    |
| Qwen3.5-4B + LoRA fine-tuning                                 | 1000    | 0.1935 | 0.5211  | 4.5341    |
| MedGemma-4B-it + LoRA only, mask-matched subset               | 305     | –      | –       | 4.4269    |
| MedGemma-4B-it + LoRA + visual grounding, mask-matched subset | 305     | 0.4139 | 0.6966  | 4.5751    |

Table 15: Polyp VQA evaluation on Kvasir-VQA-X1

The results show that LoRA fine-tuning is the main driver of performance improvement. MedGemma-4B-it improves from a BLEU-4 score of 0.0055 and LLM Judge score of 2.7521 to 0.3657 and 4.6351, respectively. Qwen3.5-4B also benefits strongly from LoRA, increasing from 0.0261 BLEU-4 and 3.4720 judge score to 0.1935 and 4.5341. These improvements indicate that the base VLMs do not directly match the answer style and clinical content distribution of Kvasir-VQA-X1, while lightweight task adaptation substantially improves both lexical matching and semantic answer quality.

Between the two backbones, MedGemma-4B-it achieves stronger fine-tuned performance than Qwen3.5-4B. This is consistent with the expectation that a medically oriented VLM backbone is better aligned with endoscopic visual concepts and clinical answer generation. Qwen3.5-4B remains a strong baseline after fine-tuning, but its lower BLEU-4 and ROUGE-L suggest that it may be less aligned with the dataset’s medical visual-language distribution.

The visual grounding comparison must be interpreted on the mask-matched subset only. As shown in Table 11, only 1,336 of the 6,212 training images have usable segmentation masks, reducing the training samples from 143,594 to 42,382 for the grounded setting. Therefore, the fair comparison is not between the 1000-sample MedGemma LoRA result and the visual-grounded result. Instead, the correct baseline is the MedGemma LoRA-only model evaluated again on the same 305 evaluation samples that have matched visual grounding inputs. Under this controlled comparison, visual grounding improves the LLM Judge score from 4.4269 to 4.5751. This suggests that segmentation masks provide useful spatial evidence for VQA when the mask is available and aligned with the question.

The original-image setting evaluates whether the VLM can identify relevant visual evidence without additional guidance. The visual-grounded setting evaluates whether a segmentation mask improves answer quality by focusing attention on the polyp region. Since only a subset of samples contains matched segmentation masks, the visual grounding analysis is conducted on 305 mask-matched samples.

| LLM Judge Criterion      | MedGemma LoRA only | MedGemma LoRA + visual grounding | Difference |
|--------------------------|--------------------|----------------------------------|------------|
| Factual Accuracy         | 4.2918             | 4.4951                           | +0.2033    |
| Completeness             | 4.3607             | 4.5016                           | +0.1409    |
| Clinical Relevance       | 4.8984             | 4.9246                           | +0.0262    |
| No Hallucination         | 4.7607             | 4.8393                           | +0.0786    |
| Weighted Score           | 4.4269             | 4.5751                           | +0.1482    |
| Samples in 4.0–5.0 range | 238/305            | 255/305                          | +17        |

Table 16: LLM Judge comparison between LoRA-only and visual-grounded MedGemma on the mask-matched subset

The criterion-level judge results in Table 16 indicate that visual grounding improves all four judge criteria. The largest gain appears in factual accuracy (+0.2033), followed by completeness (+0.1409). This pattern is important: the mask does not merely make the generated answer

more fluent, but appears to help the model select more correct visual evidence and include more of the expected clinical content. The smaller gain in clinical relevance (+0.0262) is also reasonable because the LoRA-only model already has a very high clinical relevance score of 4.8984, leaving little room for improvement.

The hallucination score also improves from 4.7607 to 4.8393. This suggests that explicit spatial grounding may reduce the tendency to add unsupported visual claims. In medical VQA, this is valuable because a confident but visually unsupported answer can be unsafe even when it uses correct clinical terminology. The distribution of weighted scores also improves: the number of answers in the 4.0–5.0 range increases from 238 to 255 out of 305 samples. Therefore, the improvement is not only reflected in the average score, but also in a larger number of high-quality answers.

The controlled comparison in Table 15, together with the criterion-level breakdown in Table 16, supports the design choice of connecting the Segmentation Agent with the Polyp VQA Agent. The segmentation mask acts as an intermediate visual evidence signal, allowing the VQA model to focus on the lesion area instead of relying only on global image context or dataset priors. However, the result should not be interpreted as proof that visual grounding is always beneficial. The grounded setting depends on mask availability and mask quality. If the segmentation model misses the lesion, highlights irrelevant tissue, or assigns an incorrect region, the same grounding mechanism may guide the VQA model toward the wrong evidence. For this reason, visual grounding should be treated as a useful but conditional intervention, and future evaluation should include failure cases where incorrect masks degrade the answer.

## 9 Chapter 9. Overall Discussion and Limitations

### 9.1 Strengths of the Proposed System

The main strength of the system is its modular AI architecture. Instead of relying on a single model, the system combines specialized agents for text reasoning, graph reasoning, retrieval, visual analysis, VQA, memory, and validation. This design improves extensibility and makes the reasoning process easier to analyze.

The architecture also supports complementary reasoning sources. RAG provides explanatory text, KG provides structured relations, Web Search provides fallback coverage, and memory provides patient-specific continuity. In the Vision Module, segmentation provides localized evidence while the VQA agent converts visual and textual input into an answer. This combination allows the system to handle a wider range of medical-assistant tasks than a single text-only chatbot.

Another strength is that the system makes several uncertainty points explicit. MedGemma CoT-based sufficiency checks identify incomplete symptom descriptions after KG/RAG agents have selected relevant context. Routing determines whether a specialized agent is appropriate. Visual grounding is evaluated as a separate input setting rather than assumed to be beneficial. Memory is treated as patient-reported context rather than verified diagnosis. These design choices improve transparency and make the system easier to evaluate.

### 9.2 Text Module Limitations

The Text Module depends on the coverage and quality of its data sources. RAG may retrieve irrelevant documents, and the Knowledge Graph may not contain all relevant medical relations. Multi-hop QA also depends on correctly identifying missing information.

Vietnamese medical language adds additional challenges. Users may describe symptoms in informal language, use regional terms, or mix Vietnamese with English medical terminology. If



query expansion does not capture these variations, retrieval quality may decrease. If expansion is too broad, the system may retrieve clinically unrelated contexts. This trade-off affects both answer accuracy and safety.

The Text Module also cannot guarantee clinical correctness. Even when KG and RAG provide useful evidence, the final synthesis is still generated by an LLM and may omit caveats or overstate uncertainty. Human validation and expert review remain necessary for high-risk cases.

### 9.3 Vision Module Limitations

The Vision Module is limited by the quality of segmentation and VQA fine-tuning data. General medical image interpretation is broad and difficult to validate across all modalities. Polyp VQA is more focused but depends on the relevance of the training dataset and the quality of segmentation masks.

Segmentation errors can propagate to downstream VQA. If the mask misses the polyp region, highlights irrelevant tissue, or assigns the wrong class, the grounded VQA input may mislead the model. Therefore, visual grounding is not automatically safer than original-image VQA. It should be evaluated as an intervention that can help in some cases and harm in others.

The General Medical Image Agent has broader coverage but weaker guarantees. Without modality-specific fine-tuning and evaluation, its outputs should remain descriptive and cautious. This agent is useful for system extensibility, but the thesis does not claim comprehensive diagnostic performance across all medical image types.

### 9.4 Kvasir-VQA-X1 Limitations and Domain Transfer

Kvasir-VQA-X1 is useful for endoscopic VQA, but dataset coverage and annotation style may not fully match real deployment images or user questions. Metrics such as BLEU and ROUGE-L may not capture clinical correctness, so LLM Judge and qualitative analysis are also needed.

Domain transfer is a central concern. A model fine-tuned on Kvasir-VQA-X1 may learn the style and distribution of that dataset but still struggle with images from different devices, lighting conditions, bowel preparation quality, or clinical populations. User questions in deployment may also be longer, less structured, and more ambiguous than dataset questions. These differences should be considered when interpreting evaluation results.

The two VQA settings also introduce different risks. The original-image setting depends on the VLM’s own attention and visual reasoning. The grounded setting depends on both segmentation quality and the VLM’s ability to use the overlay correctly. A fair discussion must therefore analyze not only which setting scores higher but also why it succeeds or fails.

### 9.5 Memory Layer Limitations

Memory can be stale, incomplete, or incorrectly extracted. Patient-reported facts may change over time. Therefore, memory should be used as supporting context only and should not override current user input or medical evidence.

A memory layer can also introduce privacy and governance concerns. Even clinically useful facts may be sensitive. The system should avoid storing unnecessary personally identifiable information and should support correction or deletion of stored facts. In the current thesis scope, memory is evaluated as a system integration feature rather than as a complete clinical record management system.

Another limitation is that memory retrieval may surface irrelevant old facts. If the response

generator uses such facts without checking relevance, personalization can become distracting or unsafe. The agent should only use memory when it is directly related to the current question.

## 9.6 Evaluation and Safety Limitations

The evaluation is component-oriented and system-oriented, but it is not a clinical trial. The system should be positioned as medical decision support, not as a replacement for physicians. More rigorous expert evaluation is necessary before any clinical use.

The metrics used in this thesis capture important but incomplete aspects of performance. Dice Score measures segmentation overlap but does not fully measure clinical usefulness. BLEU and ROUGE-L measure text similarity but not necessarily medical correctness. LLM Judge can evaluate semantic quality but may contain evaluator bias. One-Hop Accuracy and Multi-Hop Awareness Score measure reasoning behavior but cannot cover all clinical scenarios.

For these reasons, the results should be interpreted as evidence of technical feasibility and comparative system behavior, not as proof of clinical readiness. Future evaluation should include medical expert review, larger datasets, prospective testing, and safety analysis under realistic user interactions.

# 10 Chapter 10. Conclusion and Future Work

## 10.1 Summary of Achievements

This thesis demonstrates that a medical assistant can be made more controllable by separating routing, evidence retrieval, visual grounding, memory retrieval, and response generation into specialized agents. The resulting system improves observability because intermediate evidence can be inspected before final answer generation. It also improves safety behavior by distinguishing evidence availability from clinical sufficiency, preserving patient context without treating it as verified diagnosis, and evaluating visual grounding as a conditional intervention rather than an assumed benefit.

Experimentally, the thesis shows four main outcomes. First, structured Knowledge Graph evidence improves relation-oriented text reasoning compared with text-only generation. Second, connected-component and color-dominance post-processing improves segmentation consistency for colonoscopy polyp masks. Third, LoRA fine-tuning substantially improves Polyp VQA performance for both MedGemma-4B-it and Qwen3.5-4B. Fourth, segmentation-grounded VQA improves judge-based answer quality on the mask-matched subset, especially in factual accuracy and completeness. Together, these results support the thesis argument that modular grounding and evidence control can improve medical-assistant behavior at the system level.

## 10.2 Technical Contributions

The technical contributions are:

- a graph-orchestrated multi-agent architecture that separates routing, text reasoning, vision reasoning, memory retrieval, and validation;
- a Text Module with Multi-hop QA, KG/RAG evidence construction, MedGemma CoT sufficiency assessment, and fallback search;
- a Vision Module that links polyp segmentation with segmentation-grounded Polyp VQA;
- a region-level post-processing method for improving multi-class polyp mask consistency;
- LoRA fine-tuned Polyp VQA models based on MedGemma-4B-it and Qwen3.5-4B, evaluated under original-image and visual-grounded settings;
- a patient-scoped memory layer and human-in-the-loop validation mechanism for context-

tual continuity, feedback, and review.

### **10.3 Future Work**

Future work should focus on expanding Vietnamese medical datasets, improving Knowledge Graph coverage, strengthening RAG retrieval and reranking, improving visual grounding robustness, refining LoRA fine-tuning for VQA, adding expert validation, and developing stronger memory governance mechanisms. Clinical evaluation with medical professionals is also necessary before any real-world deployment.

## References

- [1] LangGraph Documentation - Build language agents as graphs. <https://langchain-ai.github.io/langgraph/>
- [2] Google AI. Gemini 2.0 Flash: A new generation of multimodal AI. <https://deepmind.google/technologies/gemini/flash/>
- [3] Qdrant Documentation - Vector Database for AI Applications. <https://qdrant.tech/documentation/>
- [4] Chhikara, P., Khant, D., Aryan, J., Singh, T., Yadav, R. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [5] Mem0 Documentation - Overview and Memory Operations. <https://docs.mem0.ai/>
- [6] Tavily AI - Real-time search API for AI agents. <https://tavily.com/>
- [7] Wang, L., Yang, N., Huang, X., et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- [8] Nguyen, T.M., Le, V.H., Phan, X.H. VimedAQA: A Vietnamese Medical Question Answering Dataset. *Proceedings of the 12th International Conference on Knowledge and Systems Engineering (KSE)*, 2023.
- [9] Lewis, P., Perez, E., Piktus, A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 2020.
- [10] Yu, K.H., Beam, A.L., Kohane, I.S. Artificial intelligence in healthcare. *Nature Biomedical Engineering*, vol. 2, no. 10, pp. 719-731, 2018.
- [11] Zhang, Y., Chen, H., Song, Y., et al. Multimodal machine learning for medical image analysis: A comprehensive review. *Medical Image Analysis*, vol. 78, 102384, 2022.
- [12] Singhal, K., Azizi, S., Tu, T., et al. Large language models encode clinical knowledge. *Nature*, vol. 620, pp. 172-180, 2023.
- [13] Singhal, K., Tu, T., Gottweis, J., et al. Towards Expert-Level Medical Question Answering with Large Language Models. *arXiv preprint arXiv:2305.09617*, 2023.
- [14] Wu, C., Lin, W., Zhang, X., Zhang, Y., Wang, Y., Xie, W. PMC-LLaMA: Towards Building Open-source Language Models for Medicine. *arXiv preprint arXiv:2304.14454*, 2023.
- [15] Labrak, Y., Bazoge, A., Morin, E., Gourraud, P.A., Rouvier, M., Dufour, R. BioMistral: A Collection of Open-Source Pretrained Large Language Models for Medical Domains. *arXiv preprint arXiv:2402.10373*, 2024.
- [16] Li, C., Wong, C., Zhang, S., et al. LLaVA-Med: Training a Large Language-and-Vision Assistant for Biomedicine in One Day. *Advances in Neural Information Processing Systems*, 2023.
- [17] Moor, M., Huang, Q., Wu, S., et al. Med-Flamingo: a Multimodal Medical Few-shot Learner. *arXiv preprint arXiv:2307.15189*, 2023.

- [18] Tonmoy, S.M.T.I., Zaman, S.M.M., Jain, V., et al. A Comprehensive Survey of Hallucination Mitigation Techniques in Large Language Models. *arXiv preprint arXiv:2401.01313*, 2024.
- [19] Jin, Q., Dhingra, B., Liu, Z., Cohen, W.W., Lu, X. PubMedQA: A Dataset for Biomedical Research Question Answering. *Proceedings of EMNLP-IJCNLP*, 2019.
- [20] Jin, D., Pan, E., Oufattole, N., Weng, W.H., Fang, H., Szolovits, P. What Disease does this Patient Have? A Large-scale Open Domain Question Answering Dataset from Medical Exams. *Applied Sciences*, vol. 11, no. 14, 2021.
- [21] Pal, A., Umapathi, L.K., Sankarasubbu, M. MedMCQA: A Large-scale Multi-Subject Multi-Choice Dataset for Medical Domain Question Answering. *Conference on Health, Inference, and Learning*, 2022.
- [22] Yang, Z., Qi, P., Zhang, S., et al. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. *Proceedings of EMNLP*, 2018.
- [23] Liu, X., Faes, L., Kale, A.U., et al. A comparison of deep learning performance against health-care professionals in detecting diseases from medical imaging. *Nature Medicine*, vol. 25, no. 8, pp. 1252-1258, 2019.
- [24] Bickmore, T.W., Gruber, A., Picard, R. Establishing the computer-patient working alliance in automated health behavior change interventions. *Patient Education and Counseling*, vol. 59, no. 1, pp. 21-30, 2005.
- [25] Nguyen, D.Q., Nguyen, A.T. PhoBERT: Pre-trained language models for Vietnamese. *Findings of EMNLP*, 2020.
- [26] Laranjo, L., Dunn, A.G., Tong, H.L., et al. Conversational agents in healthcare: a systematic review. *Journal of the American Medical Informatics Association*, vol. 25, no. 9, pp. 1248-1258, 2018.
- [27] Tampuu, A., Matiisen, T., Kodelja, D., et al. Multiagent cooperation and competition with deep reinforcement learning. *PloS one*, vol. 12, no. 4, e0172395, 2017.
- [28] Nogueira, R., Cho, K. Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.
- [29] Ronneberger, O., Fischer, P., Brox, T. U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.
- [30] Zhou, Z., Siddiquee, M.M.R., Tajbakhsh, N., Liang, J. UNet++: A Nested U-Net Architecture for Medical Image Segmentation. *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*, pp. 3-11, 2018.
- [31] Chen, L.C., Zhu, Y., Papandreou, G., Schroff, F., Adam, H. Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation. *European Conference on Computer Vision (ECCV)*, 2018.
- [32] Fan, D.P., Ji, G.P., Zhou, T., Chen, G., Fu, H., Shen, J., Shao, L. PraNet: Parallel Reverse Attention Network for Polyp Segmentation. *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2020.

- [33] Jha, D., Smedsrud, P.H., Riegler, M.A., Halvorsen, P., de Lange, T., Johansen, D., Johansen, H.D. Kvasir-SEG: A Segmented Polyp Dataset. *International Conference on Multimedia Modeling*, 2020.
- [34] Dong, B., Wang, W., Fan, D.P., Li, J., Fu, H., Shao, L. Polyp-PVT: Polyp Segmentation with Pyramid Vision Transformers. *arXiv preprint arXiv:2108.06932*, 2021.
- [35] Rahman, M.M., Munir, M., Marculescu, R. EMCAD: Efficient Multi-scale Convolutional Attention Decoding for Medical Image Segmentation. *arXiv preprint arXiv:2405.06880*, 2024.
- [36] Li, Y., Hu, M., Yang, X. Polyp-SAM: Transfer SAM for Polyp Segmentation. *arXiv preprint arXiv:2305.00293*, 2023.
- [37] Gautam, S., Storås, A., Midoglu, C., Hicks, S.A., Thambawita, V., Halvorsen, P., Riegler, M.A. Kvasir-VQA: A Text-Image Pair GI Tract Dataset. *arXiv preprint arXiv:2409.01437*, 2024.
- [38] Gautam, S., Riegler, M.A., Halvorsen, P. Kvasir-VQA-x1: A Multimodal Dataset for Medical Reasoning and Robust MedVQA in Gastrointestinal Endoscopy. *arXiv preprint arXiv:2506.09958*, 2025.
- [39] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations (ICLR)*, 2022.
- [40] Sellergren, A., Kazemzadeh, S., Jaroensri, T., et al. MedGemma Technical Report. *arXiv preprint arXiv:2507.05201*, 2025.
- [41] Yang, A., Li, A., Yang, B., et al. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.